Applying Black-Box Testing Techniques to Two Case Studies

Mai Hải Thành[a]

[a]Independent Research; FIT-UET-VNU, Hanoi, Vietnam

October 2026

Abstract

Decision table testing and boundary value analysis are standard black-box techniques, but each answers a different question: the decision table says which combinations of conditions must be tested, and boundary value analysis says which inputs to use inside each combination. This report applies both, together with mutation analysis, to two specifications with multi-variable behaviour: an ATM withdrawal validator with five inputs and a fixed rule precedence, and a quadratic equation classifier whose central boundary is the discriminant $\Delta = b^2 - 4ac = 0$. For each specification the conditions are written as an extended-entry decision table, infeasible columns are proved infeasible, the table is collapsed and checked by rule count, and a test suite is built in four layers: one test per rule, 3-value boundary value analysis on every ordered condition, an explicitly labelled extension (boundary combinations and don't-care expansion), and tests added in response to mutation analysis. Both designs were implemented in C#, executed, and evaluated with designed mutants and with mutants generated by Stryker.NET, cumulatively per layer and, for the ATM, against a size-matched random baseline. In these two small case studies the results are as follows. In the ATM, one test per rule killed 3 of 13 designed and 14 of 22 tool-generated mutants, and boundary values raised this to 10 and 21; the same number of random tests from the same rules killed on average 0.06 of the seven designed mutants that the boundary values add. The two modelled precedence reversals were killed only by the don't-care expansion, and a wrong modulus of 25,000 survived the whole suite. In the quadratic solver, one test per rule already killed all ten designed mutants with the chosen nominal inputs and the boundary tests added no kill; neither mutant population, however, contains a relational fault at the boundaries $b = 0$ and $c = 0$, at which these tests are aimed, and two such faults, executed separately, were killed only by boundary tests.

Keywords: black-box testing; decision table testing; boundary value analysis; mutation testing; test design

Contents

1 Introduction **3**

2 Theoretical Background **4**

2.1 Decision Table Testing 4
2.2 Boundary Value Analysis 5
2.3 Nominal Values 5
2.4 Multi-Variable Boundaries 6
2.5 Mutation Testing 6
2.6 Related Work and Positioning 6
2.7 Notation 7

3 Methodology **7**
3.1 From Specification to Domain . . . 8
3.2 Decision Table Construction . . . 8
3.3 Boundary Identification . . . 8
3.4 Test Suite Construction . . . 8
3.5 Mutation Analysis . . . 9
3.6 Executed Validation . . . 9

4 Case Study I: ATM Withdrawal Validation **10**
4.1 Problem Context . . . 10
4.2 Formal Specification . . . 11
4.3 Input and Output Domains . . . 12
4.4 Constraints and Dependencies . . . 12
4.5 Nominal Values . . . 12
4.6 Decision Table . . . 12
4.7 Condition Regions . . . 14
4.8 Boundary Value Analysis . . . 15
4.9 Boundary Combinations and Don't-Care Expansion . . . 17
4.10 Test Suite Construction . . . 18
4.11 Mutation Model . . . 21
4.12 Test–Mutant Analysis . . . 22
4.13 Executed Validation . . . 24
4.14 Case Study Discussion . . . 27

5 Case Study II: Quadratic Equation Solver **27**
5.1 Problem Context . . . 27
5.2 Formal Specification . . . 27
5.3 Input and Output Domains . . . 28
5.4 Constraints and Dependencies . . . 28
5.5 Nominal Configurations . . . 28
5.6 Decision Table . . . 29
5.7 Condition Regions . . . 30
5.8 Boundary Value Analysis . . . 31
5.9 Boundary Combinations and Don't-Care Expansion . . . 33
5.10 Test Suite Construction . . . 33
5.11 Mutation Model . . . 35
5.12 Test–Mutant Analysis . . . 36
5.13 Executed Validation . . . 38
5.14 Case Study Discussion . . . 40

6 Discussion **41**
6.1 Comparison of the Two Case Studies . . . 41
6.2 Lessons Common to Both Case Studies . . . 42

7 Limitations **43**

8 Conclusion **44**

1 Introduction

A black-box test suite has to stand in for an input domain that is far larger than the suite. Test design techniques make that substitution defensible by deriving each test from the specification rather than choosing it ad hoc. Two techniques are particularly common for rule-based specifications. *Decision table testing* records how combinations of conditions determine the system's actions and requires every feasible combination to be tested [3, 4, 15]. *Boundary value analysis* (BVA) selects inputs at and next to the points where a condition changes its truth value, because developers are more likely to make errors at those values [1, 2, 15].

The two techniques are complementary by construction: the table supplies the rules, BVA the inputs inside them. The coverage criterion of a decision table names the combinations to be tested but not the input to use within a combination; a test taken from the interior of a rule can pass even when the rule's boundary is implemented wrongly. Conversely, BVA applied variable by variable does not consider combinations, and in specifications whose conditions relate several inputs (a balance compared with an amount plus a fee, or a discriminant computed from three coefficients) a boundary chosen for one variable in isolation is generally not a boundary of the condition that decides the output. A further gap concerns the don't-care entries of a collapsed decision table: they encode which condition is evaluated first, yet the standard coverage criterion can be met without ever observing that order.

This report addresses the following research questions on two case studies:

RQ1. How are the conditions, rules and infeasible combinations of a decision table derived for a specification whose conditions depend on several inputs, and how is the completeness of the table checked?

RQ2. Which test inputs does 3-value BVA add to decision table coverage, and which faults do those inputs detect that one test per rule does not?

RQ3. Which faults require tests beyond the coverage criterion of the decision table (boundary combinations, don't-care expansion, mutation feedback), and how large is their contribution?

Scope. The core of the study is decision table testing and 3-value BVA (layers L1 and L2 of §3.4). Everything else is an extension and is labelled as such where it occurs: the refinements of the minimal decision tables, layer L3 (boundary combinations and don't-care expansion), layer L4 (mutation feedback), the mutation analysis itself, and the executed validation.

The two case studies differ in structure but are treated with the same method:

(1) **ATM Withdrawal Validation** (Section 4) – a validator with five inputs, mixed numeric and categorical domains, a fixed rule precedence, and a fee that couples a categorical input to a numeric threshold.

(2) **Quadratic Equation Solver** (Section 5) – a classifier over $a, b, c \in \mathbb{Z}$ whose behaviour is decided by the case split $a = 0$ and by the sign of the derived quantity $\Delta = b^2 - 4ac$.

The report is first a test-design document: every condition, rule, boundary, test case and mutant in the analytical subsections of Sections 4 and 5 is derived from the stated specification, and the mutant-killing claims there are labelled as *analytically expected*. Each case study then ends with an executed validation (§4.13, §5.13): the specification is implemented, every designed test is executed, the decision table is executed as an oracle, the analytical mutation matrix is compared with the kill sets of executable mutant models, and an automated mutation tool measures a mutation score on a mutant population that was not designed together with the suites. Apart from two cells of the quadratic Test×Mutant matrix, which show executed values

and are marked as such, the analytical subsections use no measured number; where they mention an executed result, they refer forward to the executed validation.

Every test case in this report is traceable to a decision rule, a boundary coverage item, a boundary combination or further point on a derived boundary, a don't-care expansion, or a mutant that no earlier test kills. Section 2 defines the techniques and positions the report with respect to the sources (§2.6); Section 3 describes the method in general terms; Sections 4 and 5 apply it; Section 6 compares the two case studies and draws the lessons common to both; Section 7 states the limitations and Section 8 answers RQ1–RQ3.

2 Theoretical Background

2.1 Decision Table Testing

A decision table records how combinations of conditions determine the actions a system takes [3, 4, 15]. Its rows are *conditions* (predicates over the inputs) and *actions* (the responses the specification prescribes); each column is a *rule*, which assigns one entry to every condition and names the actions that follow. The notation follows the ISTQB syllabus [15, §4.2.3]: T/F for a satisfied or unsatisfied condition and "−" for a condition whose value is irrelevant to the rule's outcome (a *don't-care* entry). The syllabus uses "N/A" for a condition that is infeasible for a rule; this report needs a different entry, "n.e." (not defined), for a condition that is not defined in a rule's case (the sign of Δ exists only when $a \neq 0$). A don't-care entry, in contrast, belongs to a condition that is defined but whose value cannot change the rule's actions. In a *limited-entry* table every condition entry is Boolean; in an *extended-entry* table a condition may take one of several values, for example a range of numbers or an equivalence partition of an input domain [15, §4.2.3].

Three properties of a decision table are checked explicitly in this report.

(i) **Completeness by rule count.** If condition c_j has n_j possible entries, a full table has $\prod_j n_j$ columns; when a condition is defined only in some cases, as q_4 in the quadratic solver, the product is taken per case and the cases are added. A don't-care entry in a rule stands for every entry of that condition, so a rule with don't-care entries on a set J of conditions covers $\prod_{j \in J} n_j$ columns of the full table. If the rules are complete and do not overlap, these counts sum to the number of feasible columns; a smaller sum indicates missing rules and a larger sum redundant or contradictory ones [3, Ch. 7]. The equality is necessary but not sufficient, since a gap and an overlap can cancel; non-overlap is therefore checked separately, by comparing the entries of every pair of rules and by execution.

(ii) **Feasibility.** A combination of condition entries is *infeasible* when no input satisfies all of them at once, typically because the conditions are logically dependent. Infeasible columns are removed from the table [15, §4.2.3]; in this report each removal is justified by a short proof, and, where the domain is finite, by an exhaustive count showing that no input reaches the column.

(iii) **Collapsing.** Columns that differ only in conditions that do not affect the actions are merged into one rule with a don't-care entry [15, §4.2.3]. In the ATM the don't-care entries arise from the rule precedence: once a rule has determined the outcome, the conditions evaluated after it cannot change it. In the quadratic solver the only one arises because c is irrelevant when $a = 0$ and $b \neq 0$. The result of merging every such pair of columns is the *minimal* table. This report deliberately keeps some columns with the same actions apart, so that each class of an extended-entry condition, or each value of a coefficient condition, has a rule and therefore a test of its own; where this is done it is stated, together with the size of the minimal table, and it is an extension of the technique.

The coverage items of decision table testing are the feasible columns of the (collapsed) table; full coverage requires at least one test per feasible column [15, §4.2.3]. Two consequences of this criterion shape the construction in Section 3.4. First, the criterion says which rule a test must reach but not which input within the rule to use, so a second technique is needed to choose the input; this report uses nominal values (§2.3) for the first test of each rule and boundary value analysis (§2.2) for the rest. Second, a test of a collapsed rule fixes each don't-care entry to one concrete value; the coverage criterion does not require the other values to be tried. Where a don't-care entry encodes an evaluation order, trying the other value is the only way to check the order itself; this report adds such tests and labels them as an extension of the criterion (Layer L3 in Section 3.4).

When a condition is an extended entry whose values are the equivalence classes of one input, the classes must be mutually exclusive and must cover the input's domain, as for equivalence partitioning [1, 15]; this is what makes the rule count in (i) meaningful. In this report every extended-entry condition is defined in set notation and both properties are checked.

2.2 Boundary Value Analysis

Boundary Value Analysis (BVA) [1, 2, 15] applies to a condition whose values are ordered, so that its true and false regions are separated by boundary points. Specifications are mostly written with relations ($<$, $\leq$, $=$, $\geq$, $>$), and the typical defect is a boundary implemented one step away from its intended position or omitted [15, §4.2.2]: an error of $\leq$ for $<$ is invisible to an input far from the boundary and visible at the boundary itself.

This report uses *3-value BVA* [15, §4.2.2]: for each boundary value v of a condition, the coverage items are v and both of its neighbours. Applied to a closed range $[\mathrm{Min}, \mathrm{Max}]$, this gives the values

$$\mathrm{Min}^-,\ \mathrm{Min},\ \mathrm{Min}^+,\ \mathrm{Max}^-,\ \mathrm{Max},\ \mathrm{Max}^+,$$

which, together with a nominal value Norm, are the values of *robust* boundary value testing [3, Ch. 5]. A condition with a single boundary T (e.g. $x \geq T$) has the coverage items $T-1$, T, $T+1$ on an integer domain. Following [3, Ch. 5], boundary tests are built under the *single-fault assumption*: one condition is placed at or next to its boundary while every other condition keeps the entry it has in the nominal test of the same rule. Tests in which two or more conditions are at their boundaries at the same time are built separately (§2.4).

Neighbours must be reachable. "Neighbour" means the nearest value the condition can actually take, which is not always $v \pm 1$. If the valid values of an input form a lattice (all multiples of a step s), the neighbours of a lattice endpoint on the lattice are $v \pm s$, and the integers $v \pm 1$ are neighbours on the integer line that fail the lattice condition; both kinds are listed, because they belong to different conditions. Likewise, a derived quantity may skip integer values, in which case its nearest reachable values have to be found before any boundary test is written (§5.8). Choosing an unreachable neighbour yields a test case that cannot be executed.

2.3 Nominal Values

A nominal (Norm) value of a condition entry is a value from the interior of that entry, away from every boundary, so that it represents the entry's ordinary behaviour [3, Ch. 5]. Nominal values serve two purposes in this report: they are the inputs of the one-test-per-rule layer, and they fill the conditions that are not under analysis in a boundary test, so that only the condition under analysis is at its edge. Some rules have no interior: a rule with a single member, or a rule defined by an equality such as $\Delta = 0$, has every member on its boundary. For such rules a nominal value does not exist; this is stated where it occurs and a canonical member of the rule is used instead. When a specification has several structurally different valid behaviours, as the quadratic solver has, one nominal tuple cannot represent all of them, and a nominal input is defined per rule (§5.5).

2.4 Multi-Variable Boundaries

Many specifications decide behaviour by a *relation among several variables*, e.g. *balance* $\geq$ *amount* + *fee* or the sign of $\Delta = b^2 - 4ac$. Such a relation is called a *derived boundary* in this report. In a decision table it appears as one condition, and its boundary values are values of the derived quantity, not of any one input. Two consequences follow. First, a boundary test of a derived condition has to be constructed by solving the relation for one input given the others; varying one input over its own domain boundaries while holding the others at nominal values does not, in general, reach the derived boundary at all. Second, several conditions can be at their boundaries at the same time, and a categorical input can move the boundary of a numeric condition (the fee selected by *isInternal* moves the balance threshold). A *boundary combination* is a test in which at least two conditions are at or next to their boundaries at once, or in which a change of one condition moves the threshold of another while the numeric inputs stay on that threshold (§4.9, §5.9). Because a derived boundary is a surface rather than a point, a single boundary test samples only one of its points; further points of the same surface, reached through other combinations of inputs, are a third kind of test considered in this report.

2.5 Mutation Testing

Mutation testing [5, 6] evaluates the fault-detection strength of a test suite. A *mutant* is the specification, or an implementation of it, with one small fault seeded into it: a relational operator swapped ($\geq \to >$), a boundary constant perturbed (20,000,000 $\to$ 19,999,999), a logical connective changed (AND $\to$ OR), or an arithmetic term altered ($b^2 - 4ac \to b^2 - 2ac$). A test *kills* a mutant if the mutant's output for the test input differs from the specified output; a mutant *survives* a suite if no test in the suite kills it. The mutation score is

$$\text{Mutation Score} = \frac{\text{killed non-equivalent mutants}}{\text{all non-equivalent mutants}} \times 100\,\%,$$

where an *equivalent mutant* computes the same output as the original for every input of the domain, so that no test can kill it, and is excluded from the denominator. In the analytical subsections of Sections 4 and 5, kill claims are **analytically expected**: they are derived by substituting each test input into the specification and into the mutant. Measured results, obtained by executing mutants, are reported separately in §4.13 and §5.13.

2.6 Related Work and Positioning

What the sources establish. Decision table testing and boundary value analysis are standard black-box techniques with stated coverage criteria. The ISTQB syllabus defines the coverage items of decision table testing as the feasible columns of the table and those of BVA as the boundary values of ordered partitions and, in the 3-value variant, their neighbours [15, §§4.2.2–4.2.3]. Jorgensen and DeVries describe decision-table-based testing, including don't-care entries and the rule count, and several forms of boundary value testing: normal, robust, and *worst-case*, in which all combinations of the boundary values of several variables are tested [3, Ch. 5, Ch. 7]. Copeland presents boundary value testing and decision table testing as separate techniques [4, Ch. 4, Ch. 5]. Two older methods combine input classes with conditions: cause-effect graphing derives a decision table from the causes and effects of a specification and complements it with boundary analysis [1], and the category-partition method derives test frames from categories of input values and constraints between them [11]. Boundaries of predicates over several variables are the subject of domain testing, which selects ON points on a border and OFF points next to it [10] [4, Ch. 8]. For evaluating suites, mutation analysis [5, 6] is supported by evidence that the detection of generated mutants correlates with the detection of real faults [7, 8]; the correlation, however, becomes weak once suite size is controlled [9], equivalent mutants must be identified, and a mutant population reflects the operators that produced it [6].

What they leave open. In these sources decision tables and BVA are presented as separate techniques, each with its own coverage criterion [15, §§4.2.2–4.2.3] [4, Ch. 4, Ch. 5]. The decision table criterion names the columns to exercise but not the input to choose within a column, and the ISTQB definition admits equivalence partitions as condition values without prescribing which member to test [15, §4.2.3]. Domain testing treats multi-variable borders, but not the rules and evaluation order of a decision table, and a collapsed table records an evaluation order only implicitly, in its don't-care entries.

Where this report sits. This report does not propose a new technique. It applies the standard techniques together on two worked case studies and makes their interaction explicit: BVA is applied to the conditions of the decision table, including derived conditions over several inputs, inside the rules in which each condition decides the outcome; the boundary tests of derived conditions are ON and OFF points in the sense of domain testing; boundary combinations resemble worst-case boundary value testing in that several conditions are placed at boundaries together, but they use the thresholds of derived conditions rather than products of single-variable boundary values; and the don't-care entries are expanded where they encode an evaluation order. The contribution of each of these steps is then measured on designed and on tool-generated mutants, and against a random baseline of the same size, so that the claims about what each layer adds rest on observed kills rather than on the definitions alone. Reid [12] compared boundary value analysis empirically with equivalence partitioning and random testing; the size-matched random baseline of this report is a comparison of the same kind.

2.7 Notation

Table 1 collects the symbols and identifiers used in the rest of the report.

Table 1: Notation

Symbol	Meaning
R1–R5	business rules of the ATM (§4.2)
Δ	discriminant $b^2 - 4ac$ of the quadratic case study
D_{out}	output domain of a case study
T, F	condition satisfied / not satisfied; for *isInternal*, T = internal, F = external
$-$, n.e.	don't-care entry; condition not defined in the rule's case (§2.1)
a_1, a_2	action stubs of a decision table
$c_1, \ldots, c_5$; $q_1, \ldots, q_4$	conditions of the ATM and the quadratic decision table
$A_1, \ldots, A_4$	entries (classes) of the amount condition c_2
DRk, QRk	rules of the ATM and the quadratic decision table
T_3, T_5	thresholds of c_3 (20,000,000) and c_5 ($amount + fee$)
Norm; Min, Max; x^-, x^+	nominal value; ends of a range; neighbours below and above a value x
L1, L2, L3 (L3a, L3b), L4	construction layers of a suite (§3.4)
ACk, AXk; QCk	ATM boundary combinations and don't-care expansions; quadratic combinations
TC-ATM-nn, TC-QE-nn	designed test cases
SUP-ATM-nn, SUP-QE-nn	supplementary tests (§3.6)
Mk, Nk	designed mutants of the ATM and the quadratic solver
ASk, QSk	assumptions of this report for the ATM and the quadratic solver

3 Methodology

Both case studies follow the same five design stages (§§3.1–3.5), and both designs are then validated by a sixth, executed stage (§3.6).

3.1 From Specification to Domain

Each case study begins with an informal problem statement, which is first made precise as a *formal specification*: a function signature (named inputs with types), a set of explicit constraints (inequalities, equalities, categorical memberships), and an explicit, enumerated output domain D_{out}. Every input's domain D_i is stated explicitly (e.g. $amount \in \mathbb{Z}$, $accountStatus \in \{\text{Active}, \text{Frozen}\}$), and every assumption the problem statement leaves open is stated as such.

3.2 Decision Table Construction

Each business rule or case distinction of the specification becomes a condition of a decision table, and each element of D_{out} becomes an action. A condition over a single input with more than two behaviourally different regions (e.g. an amount that can be too small, valid, malformed or too large) is written as an extended-entry condition whose entries are those regions, each defined in set notation and checked to be mutually exclusive and to cover the input's domain. Conditions over several inputs (e.g. $dailyUsed + amount \leq 20{,}000{,}000$, or the sign of $\Delta = b^2 - 4ac$) are kept as conditions in their own right, because they are the predicates the specification actually evaluates. The full table is then built, infeasible columns are identified and proved infeasible, columns whose outcome does not depend on a condition are collapsed into one rule with a don't-care entry, and the rule count is checked against the number of feasible columns (§2.1).

3.3 Boundary Identification

For every ordered condition of the table, the boundary values are read from the specification: the constants of its inequalities and equalities, never values introduced by the tester. For each boundary value the 3-value BVA coverage items are listed (§2.2). For conditions whose values are not dense on the integers (a lattice of valid amounts, or a discriminant that skips some integers) the neighbours are the nearest *reachable* values, established by an explicit argument. Categorical conditions have no order and therefore no boundary values; each of their entries is covered by the decision table layer.

3.4 Test Suite Construction

The suite is built in four layers, in this order. Each test is assigned to the first layer that requires it and carries a traceability record: its layer, the decision rule it exercises, the boundary or combination it targets, and its expected output.

L1 – Decision table coverage.
One test per feasible rule of the collapsed table, using the rule's nominal input. This is the coverage criterion of decision table testing [15, §4.2.3]. Where a rule has no interior (§2.3), this is stated and a canonical member of the rule is used.

L2 – Boundary values of the conditions.
3-value BVA on every ordered condition, inside each rule in which that condition decides the outcome, under the single-fault assumption. A condition evaluated in two columns with different thresholds (e.g. the balance condition with and without a fee) is analysed in each column.

L3 – Extension: combinations and don't-care expansion.
Tests that the coverage criterion of decision table testing does not require. L3a: boundary combinations and further points on a derived boundary (§2.4). L3b: don't-care expansion, in which the don't-care entry of the rule evaluated next in an evaluation order is set to its failing entry, so that the order becomes observable. This layer goes beyond the coverage criterion and is reported separately.

L4 – Mutation feedback.
Tests added only because the mutation analysis (§3.5) found a designed mutant that layers L1–L3 do not kill.

Within a layer, no two tests share their rule, boundary coverage item and combination; a coverage item already exercised by an earlier layer is not repeated as a test of its own (it may reappear as one step of a sequence whose purpose is the sequence, as in the crossing test QC1 of §5.9).

3.5 Mutation Analysis

Mutants are defined for each case study, one per plausible implementation slip suggested by the specification's own operators, constants, conditions and evaluation order. Each mutant is checked against the suite by substitution, producing a Test×Mutant matrix with the cells `K` (killed), `S` (the mutated rule is reached but the output is unchanged) and `-` (the mutated rule is not reached). A mutant that no test of layers L1–L3 kills, and that is not equivalent, is the trigger for a Layer L4 test.

3.6 Executed Validation

Stages 3.1–3.5 produce a design whose mutant-killing claims are analytically expected. The executed stage checks that design against an implementation of the specification. Both specifications were implemented in C# on .NET 10 (SDK 10.0.401, runtime 10.0.12, Windows 11). Tests are written with xUnit.net v3[1] (package `xunit.v3.mtp-v2` 4.0.1) on the Microsoft Testing Platform[2], property-based tests with FsCheck 3.4.0[3], a .NET implementation of the QuickCheck approach [13], and mutation testing uses Stryker.NET 5.0.0[4] with its Microsoft Testing Platform runner, the default mutation level, and the option `disable-bail`, so that every test is run against every mutant and the set of tests that kill each mutant is complete. With this option every mutant also runs the exhaustive quadratic sweeps; under the default time limit some of these runs exceed the limit and are reported as timeouts instead of being classified, so the quadratic runs use an additional per-mutant time limit of 60 s (`additional-timeout`), while the ATM runs use the default. No reported run contains a timeout or a compile error. The Stryker configuration files and a small program that counts the mutants killed by the designed suite in a Stryker report (`tools/StrykerDesignedKills.cs`) are part of the repository (Data and code availability). Every number reported in §4.13 and §5.13 comes from the runs of 3 October 2026 on the final test projects (clean Release build, compiler warnings treated as errors) and is asserted by a test or read from a tool report; none is estimated. The random baseline uses the fixed seed `20261002`; FsCheck draws new random cases on each run, and no reported number depends on them. The executed stage consists of six checks per case study:

(i) **Suite execution.** Each designed test case is one test (one data row, or for the crossing sequence TC-QE-28 two facts) that carries its identifier (TC-ATM-*nn*, TC-QE-*nn*) and its layer, rule, boundary and combination as test metadata, so that a run can be filtered by, e.g., layer L3 or rule DR5 alone; L3a and L3b are distinguished by the combination label (AC or AX). Supplementary tests (SUP-ATM-*nn*, SUP-QE-*nn* and a few guard tests) cover the implementation's handling of inputs outside the specified domain and values that the design does not need.

(ii) **Decision table checks.** The decision table is executed as an oracle. For the ATM, one witness input is constructed for each of the 56 feasible columns of the full table; for the

[1] https://xunit.net
[2] https://learn.microsoft.com/dotnet/core/testing/microsoft-testing-platform-intro
[3] https://fscheck.github.io/FsCheck
[4] https://stryker-mutator.io

quadratic solver, every triple of the test range is evaluated against all rules. The checks confirm that every executed input matches exactly one rule and that the implementation returns that rule's action; for the quadratic solver they also confirm that no input reaches an infeasible column.

(iii) **Matrix reproduction.** Each designed mutant is implemented in an executable reference model of the specification that injects exactly one mutant. The model is first checked to agree with the specification and the implementation on every suite row. For each mutant, the set of suite tests whose output differs from the unmutated model is then compared with the `K` cells of that mutant's column in the analytical matrix, transcribed verbatim. Execution distinguishes `K` from the other two codes; the split between `S` and `-` remains analytical.

(iv) **Layer ablation.** The designed mutants and the tool-generated mutants are evaluated against the cumulative layers, and the designed mutants also against each layer alone and against a size-matched random baseline (§4.13).

(v) **Tool-generated mutants.** Stryker.NET generates mutants from the implementation's source code with its standard operators. Unlike the designed mutants, this population was not constructed together with the suites (but see the limitations in Section 7).

(vi) **Properties and exhaustive execution.** FsCheck checks invariants and metamorphic relations [14] on 1,000 (ATM) or 2,000 (quadratic) generated cases per property; for the quadratic solver, the classifier is executed on the whole test range.

Table 2 lists the executed test groups. A "test" is one test result reported by the test platform; the groups contain tests of different kinds, so the totals only state that every test passes.

Table 2: Executed test groups (number of tests per case study)

Group	Content	ATM	Quadratic
Designed suite	TC-ATM-01–37; TC-QE-01–30 (TC-QE-28 as two sequence checks)	37	31
Supplementary	Domain guards, overflow extremes, undefined status (ATM), test-range extremes ±100 and ±101 (quadratic), further values	12	19
Decision table checks	Feasible-column witnesses (ATM); evaluation of the whole test range (quadratic)	5	2
Matrix reproduction	Model agreement per suite row, kill set per mutant column, score; M4 equivalence sweep, granularity checks and precedence transpositions (ATM); relational slips at the zero tests (quadratic)	65	48
Layer ablation	Cumulative and per-layer kill sets; L1 of the minimal table; random baseline (ATM)	4	3
Properties	Invariants and metamorphic relations (FsCheck)	8	6
Exhaustive	Every $(a, b, c) \in [-100, 100]^3$	–	2
Total		131	111

4 Case Study I: ATM Withdrawal Validation

4.1 Problem Context

An ATM must decide, for a single withdrawal request, whether to approve it or reject it, and if rejecting, for which single, well-defined reason. The decision depends on five inputs that interact:

the account's monetary state (current balance, amount already withdrawn today), the requested amount, the account's administrative state (active or frozen), and whether the transaction is being made at the bank's own ("internal") ATM network or a foreign ("external") one, which determines whether a fee is charged.

4.2 Formal Specification

$$\mathit{Withdraw}(\mathit{balance},\ \mathit{dailyUsed},\ \mathit{amount},\ \mathit{accountStatus},\ \mathit{isInternal}) \to D_{out} \tag{1}$$

with input domains

$$\mathit{balance} \in \mathbb{Z}, \quad \mathit{dailyUsed} \in \mathbb{Z}_{\geq 0}, \quad \mathit{amount} \in \mathbb{Z},$$
$$\mathit{accountStatus} \in \{\text{Active}, \text{Frozen}\}, \quad \mathit{isInternal} \in \{\text{True}, \text{False}\}.$$

Assumptions of this report. The problem statement leaves three points open; each is settled by an explicit assumption.

(AS1) Balance.
$\mathit{balance} \in \mathbb{Z}$ is an unbounded signed integer: an account may be overdrawn, and no upper bound is given. Designed test values stay within the 32-bit signed range.

(AS2) Amount withdrawn today.
$\mathit{dailyUsed} \in \mathbb{Z}_{\geq 0}$: it is a running total of the day's withdrawals, which only adds non-negative amounts to a counter that starts at 0. Negative values are outside the domain rather than a separate rejection case.

(AS3) Evaluation order.
When several rules fail, the order below decides which rejection is reported.

Business rules. R1, R2, R3 and R5 are predicates; R4 defines the fee used by R5.

$$\text{(R1) Status:} \quad \mathit{accountStatus} = \text{Active} \tag{2}$$

$$\text{(R2) Amount range \& granularity:} \quad 50{,}000 \leq \mathit{amount} \leq 20{,}000{,}000$$
$$\wedge\ \ \mathit{amount} \equiv 0 \pmod{50{,}000} \tag{3}$$

$$\text{(R3) Daily limit:} \quad \mathit{dailyUsed} + \mathit{amount} \leq 20{,}000{,}000 \tag{4}$$

$$\text{(R4) Fee:} \quad \mathit{fee} = \begin{cases} 0, & \mathit{isInternal} = \text{True} \\ 50{,}000, & \mathit{isInternal} = \text{False} \end{cases} \tag{5}$$

$$\text{(R5) Balance:} \quad \mathit{balance} \geq \mathit{amount} + \mathit{fee} \tag{6}$$

Output domain.

$$D_{out} = \{\text{Approved},\ \text{Rejected_InvalidAccount},\ \text{Rejected_InvalidAmount},\ \text{Rejected_DailyLimit},\ \text{Rejected_InsufficientBalance}\}$$

Precedence (AS3). The validator evaluates the rules in a fixed order and returns on the first failure:

$$\text{R1} \ \to\ \text{R2} \ \to\ \text{R3} \ \to\ \text{R5} \ \to\ \text{Approved}.$$

Some fixed order is required, because otherwise an input that fails several rules has no single specified output. This report chooses the order above: the account status is checked before any amount; the amount is checked before the sums that use it; and the daily limit, a policy of the bank, is checked before the customer's balance. Other orders would be equally consistent, and would change only the outputs of inputs that fail more than one rule. In the decision table (§4.6) the order appears as don't-care entries, and it is exercised by the don't-care expansion tests (§4.9), in which two adjacent rules fail at the same time.

4.3 Input and Output Domains

Table 3: ATM input domains

Input	Domain	Role
balance	$\mathbb{Z}$	Current account balance (currency units)
dailyUsed	$\mathbb{Z}_{\geq 0}$	Amount already withdrawn earlier today
amount	$\mathbb{Z}$	Requested withdrawal amount
accountStatus	$\{\text{Active}, \text{Frozen}\}$	Administrative account state
isInternal	$\{\text{True}, \text{False}\}$	Own-network vs. foreign-network transaction

D_{out} as defined above is a partition of all possible decisions into five mutually exclusive, collectively exhaustive categories; the decision table (§4.6) maps every feasible combination of conditions to exactly one of them.

4.4 Constraints and Dependencies

The rules are **not** independent single-variable checks: R4–R5 couple *isInternal* (categorical) with *amount* and *balance* (numeric) through the derived quantity *fee*; R3 couples *dailyUsed* and *amount* through their sum. The approved inputs are those for which R1, R2, R3 and R5 hold together; the decision table states this region explicitly (§4.6).

4.5 Nominal Values

Table 4: ATM nominal (Norm) configuration

Input	Nominal value	Justification
balance	25,000,000	At least $amount + fee$ for every valid amount (the largest is 20,000,000 + 50,000), so R5 holds in every test that varies another condition
dailyUsed	0	Start-of-day value. It is the minimum of $\mathbb{Z}_{\geq 0}$, but no condition has a boundary there, and it is the only value that keeps R3 satisfied for every valid amount up to 20,000,000
amount	500,000	Valid multiple of 50,000, far from both R2 endpoints
accountStatus	Active	The single valid class of R1
isInternal	True	Simpler ($fee = 0$) baseline; external transactions have their own rules, DR8 and DR9 (§4.6)

Under this tuple all rules R1–R5 hold, so the nominal case is expected to return Approved; it is the L1 test of rule DR7 (TC-ATM-07). Every other rule has its own nominal input, which changes only the inputs needed to select that rule (§4.10).

4.6 Decision Table

Conditions. The specification evaluates four predicates (R1, R2, R3, R5) and one categorical input that selects the fee (R4). R2 can fail in three different ways (amount too small, not a multiple of the step, too large), so it is written as an extended-entry condition whose four

entries are the following classes of $D_{amount} = \mathbb{Z}$:

$$A_1 = \{x \in \mathbb{Z} \mid x < 50{,}000\}, \tag{7}$$
$$A_2 = \{x \in \mathbb{Z} \mid 50{,}000 \le x \le 20{,}000{,}000 \ \wedge \ x \equiv 0 \pmod{50{,}000}\}, \tag{8}$$
$$A_3 = \{x \in \mathbb{Z} \mid 50{,}000 \le x \le 20{,}000{,}000 \ \wedge \ x \not\equiv 0 \pmod{50{,}000}\}, \tag{9}$$
$$A_4 = \{x \in \mathbb{Z} \mid x > 20{,}000{,}000\}. \tag{10}$$

A_2 and A_3 split the closed range by the modulus condition, and A_1, A_4 cover everything outside it, so the four classes are mutually exclusive and $A_1 \cup A_2 \cup A_3 \cup A_4 = \mathbb{Z}$. Only A_2 satisfies R2. Table 5 lists the five conditions and the two actions.

Table 5: ATM decision table: conditions and actions

Stub	Definition	Entries
c_1 (R1)	*accountStatus*	Active, Frozen
c_2 (R2)	class of *amount*	A_1, A_2, A_3, A_4
c_3 (R3)	*dailyUsed* + *amount* $\le$ 20,000,000	T, F
c_4 (R4)	*isInternal*	T (internal), F (external)
c_5 (R5)	*balance* $\ge$ *amount* + *fee*, with *fee* set by c_4	T, F
a_1	*fee* := 0 / *fee* := 50,000	selected by c_4
a_2	output	one element of D_{out}

Full table and feasibility. The full table has $2 \cdot 4 \cdot 2 \cdot 2 \cdot 2 = 64$ columns. Eight of them are infeasible: because *dailyUsed* ≥ 0 (AS2), every *amount* $\in A_4$ gives *dailyUsed* + *amount* $>$ 20,000,000, so $c_2 = A_4$ and $c_3 =$ T cannot hold together (2 entries of c_1 $\times$ 2 of c_4 $\times$ 2 of c_5).

Collapsing. The precedence R1 $\to$ R2 $\to$ R3 $\to$ R5 (AS3) makes every condition evaluated after the first failing one irrelevant to the actions, so those conditions become don't-care entries, and the remaining 56 feasible columns collapse into the nine rules of Table 6. Two kinds of column are deliberately not merged although their outputs agree. DR6/DR8 and DR7/DR9 differ in the action a_1 (the fee), so they are different rules in any case: c_4 changes the threshold that c_5 is evaluated against. DR2, DR3 and DR4 differ only in the entry of c_2 and have the same actions; the minimal table would merge them into one rule with the entry $A_1 \cup A_3 \cup A_4$, giving 7 rules. This report keeps them apart, so that each class of the amount has a rule and a test of its own; this refinement is an extension of the minimal table.

Table 6: ATM collapsed decision table ("$-$" = don't care; last row = number of feasible full-table columns the rule stands for)

	DR1	DR2	DR3	DR4	DR5	DR6	DR7	DR8	DR9
c_1 status	Frozen	Active	Active	Active	Active	Active	Active	Active	Active
c_2 amount	$-$	A_1	A_3	A_4	A_2	A_2	A_2	A_2	A_2
c_3 daily sum	$-$	$-$	$-$	$-^*$	F	T	T	T	T
c_4 internal	$-$	$-$	$-$	$-$	$-$	T	T	F	F
c_5 balance	$-$	$-$	$-$	$-$	$-$	F	T	F	T
a_1 fee						0	0	50,000	50,000
a_2 output	InvAcc	InvAmt	InvAmt	InvAmt	DailyLim	Insuff	Approved	Insuff	Approved
columns	28	8	8	4	4	1	1	1	1

InvAcc = Rejected_InvalidAccount, InvAmt = Rejected_InvalidAmount, DailyLim = Rejected_DailyLimit, Insuff = Rejected_InsufficientBalance. *For A_4, $c_3 =$ F is forced; the entry T is infeasible.

Rule count. DR1 stands for $4 \cdot 2 \cdot 2 \cdot 2 = 32$ columns, of which the 4 with $c_2 = A_4$, $c_3 = \mathrm{T}$ are infeasible (28); DR2 and DR3 stand for $2 \cdot 2 \cdot 2 = 8$ columns each; DR4 for 8, of which 4 are infeasible (4); DR5 for $2 \cdot 2 = 4$; DR6–DR9 for one column each. The sum $28+8+8+4+4+4 = 56$ equals the number of feasible columns $(64 - 8)$. This equality is a necessary condition for a complete table without overlapping rules (§2.1); non-overlap follows from every pair of rules having different entries in at least one condition, and the implementation is checked against the table by execution (§4.13). Every element of D_{out} is the action of at least one rule, so every output is reachable.

Approved inputs. Approval is the action of DR7 and DR9 only, i.e. it requires $c_1 = \text{Active}$, $c_2 = A_2$ and $c_3 = c_5 = \mathrm{T}$ at once, with c_5 evaluated against the threshold selected by c_4:

$$R_{\text{ATM}} = \{(\mathit{balance}, \mathit{dailyUsed}, \mathit{amount}, \mathit{accountStatus}, \mathit{isInternal}) \mid \mathit{accountStatus} = \text{Active} \ \wedge \ \mathit{amount} \in A_2 \ \wedge \ \mathit{dailyUsed} + \mathit{amount} \leq 20{,}000{,}000 \ \wedge \ \mathit{balance} \geq \mathit{amount} + \mathit{fee}(\mathit{isInternal})\}.$$

4.7 Condition Regions

The three figures below show, in this order, the four entries of c_2 on the amount line, the boundary of the derived condition c_3, and the way c_4 moves the boundary of c_5. Each is a projection of the five-dimensional input space with the remaining inputs at their nominal values (§4.5); the boundary tests on these lines are derived in §4.8 and §4.9, where the test identifiers marked in Figure 3 are also defined.

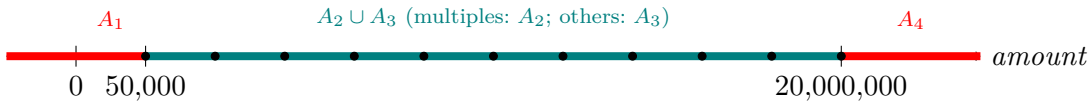


Figure 1: Entries of c_2 on the *amount* line (all other inputs held at their nominal values; schematic, not to scale). The black dots on $[50{,}000, 20{,}000{,}000]$ stand for the 400 valid multiples of 50,000 (A_2); every other integer in that interval belongs to A_3.

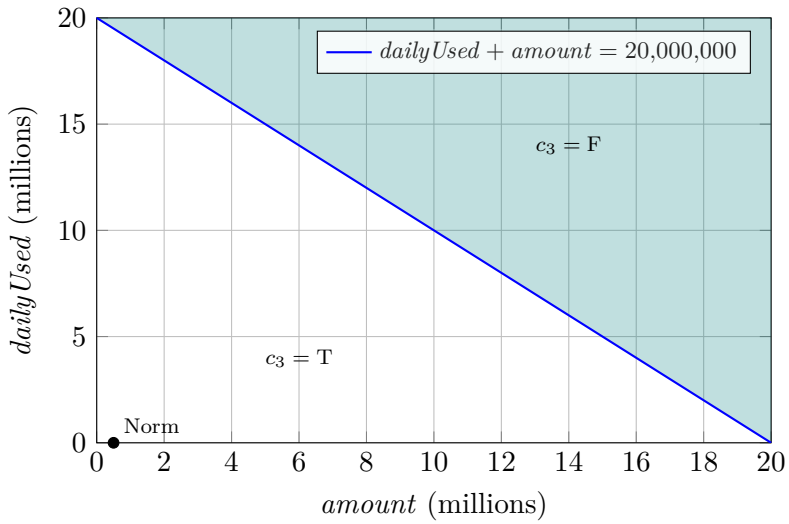


Figure 2: Daily-limit interaction: the derived boundary line $dailyUsed + amount = 20{,}000{,}000$ in the ($amount$, $dailyUsed$) plane, with $balance$, $accountStatus$, $isInternal$ fixed at their nominal values. The shaded region above the line is $c_3 = \text{F}$; the point marked Norm is the nominal input (amount 500,000, $dailyUsed = 0$). This is a 2D projection of the 5-dimensional input space.

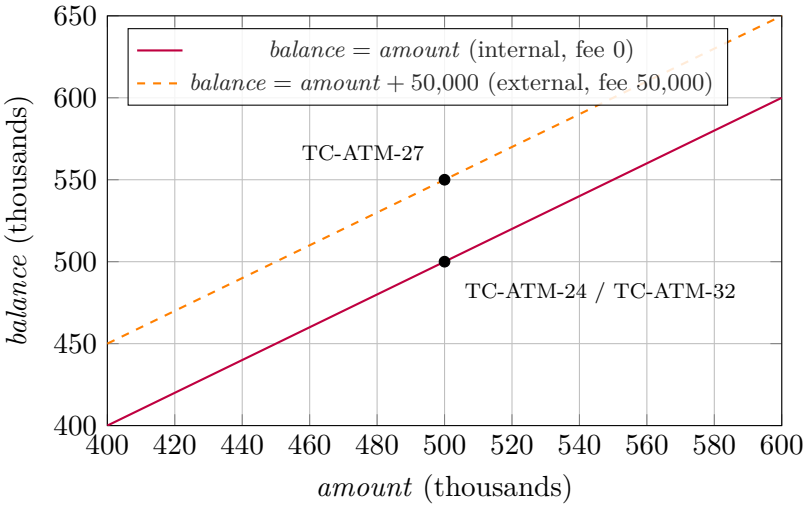


Figure 3: Balance/fee interaction: the balance threshold T_5 shifts upward by 50,000 when $isInternal$ changes from T to F ($dailyUsed = 0$, $accountStatus =$ Active). Points on or above a line satisfy c_5 for that column. The marked point (500,000; 500,000) is approved as an internal withdrawal (TC-ATM-24) and rejected as an external one (TC-ATM-32): the same numeric inputs, a different outcome, depending only on $isInternal$. The point (500,000; 550,000) lies on the external threshold and is approved as an external withdrawal (TC-ATM-27).

4.8 Boundary Value Analysis

Boundary values are taken only from the boundaries the specification states: the two ends of the amount range and its granularity (R2), the daily limit (R3) and the balance threshold (R5). c_1 and c_4 are unordered categorical conditions, so BVA does not apply to them; each of their entries is already exercised by the L1 tests of the rules in Table 6. All tests below keep every condition other than the one under analysis at the nominal entry of its rule (§4.5).

Amount (c_2). R2 is the conjunction of a range condition on the integer line and a granularity condition. The valid amounts are the lattice $A_2 = \{50{,}000, 100{,}000, \ldots, 20{,}000{,}000\}$ with step $s = 50{,}000$, so the two ends of the range have neighbours of two kinds (§2.2): integer neighbours (± 1), which leave A_2 through the granularity condition or the range condition, and lattice neighbours ($\pm s$), which are the nearest amounts that pass the granularity condition. 3-value BVA is applied on both: each end of the range gets its integer neighbours and its lattice neighbours, so that each end has five coverage items instead of three. Table 7 lists them; "Norm" is the nominal amount 500,000 of TC-ATM-07.

Table 7: 3-value BVA of c_2 (amount) with integer and lattice neighbours; balance 25,000,000, *dailyUsed* = 0, Active, internal

Item	Amount	Entry	Test	Expected / reason
Min^- (lattice)	0	A_1	TC-ATM-10	Rejected_InvalidAmount; nearest multiple of s below the range
Min^- (integer)	49,999	A_1	TC-ATM-11	Rejected_InvalidAmount; range fails (and granularity)
Min	50,000	A_2	TC-ATM-12	Approved
Min^+ (integer)	50,001	A_3	TC-ATM-13	Rejected_InvalidAmount; range holds, granularity fails
Min^+ (lattice)	100,000	A_2	TC-ATM-14	Approved
Norm	500,000	A_2	TC-ATM-07	Approved
Max^- (lattice)	19,950,000	A_2	TC-ATM-15	Approved
Max^- (integer)	19,999,999	A_3	TC-ATM-16	Rejected_InvalidAmount; granularity fails
Max	20,000,000	A_2	TC-ATM-17	Approved
Max^+ (integer)	20,000,001	A_4	TC-ATM-18	Rejected_InvalidAmount; range and granularity fail
Max^+ (lattice)	20,050,000	A_4	TC-ATM-19	Rejected_InvalidAmount; range fails, granularity holds

Three of these values need a remark. At Max = 20,000,000 the daily sum is $0 + 20{,}000{,}000$, exactly the R3 limit: the largest valid amount equals the daily limit, so no input with *amount* = Max keeps c_3 away from its boundary, and TC-ATM-17 places both c_2 (at Max) and c_3 (at T_3) on a boundary. At $\text{Min}^- = 49{,}999$ (integer) both parts of R2 fail, because 49,999 is also not a multiple of s; an implementation whose lower bound were lowered to any value below 50,000 would still reject it by granularity. At $\text{Min}^- = 0$ (lattice) the amount is a multiple of s and fails the range condition only; it is the nearest value below the range that an implementation with a lowered or missing lower bound would accept.

Daily limit (c_3). The derived condition $dailyUsed + amount \leq 20{,}000{,}000$ has one boundary, $T_3 = 20{,}000{,}000$, on the integer sum. With *amount* at its nominal value 500,000, the coverage items are reached by varying *dailyUsed* alone (Table 8). *dailyUsed* = 0, the minimum of $D_{dailyUsed}$, is not a boundary of any condition (it is the nominal value); *dailyUsed* = −1 lies outside the domain and is tested only as an implementation guard (§4.13).

Table 8: 3-value BVA of c_3; balance 25,000,000, amount 500,000, Active, internal

Item	*dailyUsed*	sum	Test	Expected
Norm	0	500,000	TC-ATM-07	Approved
$T_3 - 1$	19,499,999	19,999,999	TC-ATM-20	Approved
T_3	19,500,000	20,000,000	TC-ATM-21	Approved
$T_3 + 1$	19,500,001	20,000,001	TC-ATM-22	Rejected_DailyLimit

Balance (c_5). The threshold $T_5 = amount + fee$ depends on c_4, so c_5 has a different boundary in each c_4 column of Table 6: $T_5 = 500{,}000$ for internal and $T_5 = 550{,}000$ for external transactions at the nominal amount. Both are analysed (Table 9). R5 is non-strict, so T_5 itself is the last approved value.

Table 9: 3-value BVA of c_5 in both c_4 columns; amount 500,000, $dailyUsed = 0$, Active

Column	Item	balance	Test	Expected
internal ($T_5 = 500{,}000$)	Norm	25,000,000	TC-ATM-07	Approved
	$T_5 - 1$	499,999	TC-ATM-23	Rejected_InsufficientBalance
	T_5	500,000	TC-ATM-24	Approved
	$T_5 + 1$	500,001	TC-ATM-25	Approved
external ($T_5 = 550{,}000$)	Norm	25,000,000	TC-ATM-09	Approved
	$T_5 - 1$	549,999	TC-ATM-26	Rejected_InsufficientBalance
	T_5	550,000	TC-ATM-27	Approved
	$T_5 + 1$	550,001	TC-ATM-28	Approved

$balance \in \mathbb{Z}$ has no domain boundary, and a negative balance (an overdrawn account) is an ordinary member of the $c_5 = \mathrm{F}$ entry; it is executed as a supplementary test (§4.13).

4.9 Boundary Combinations and Don't-Care Expansion

This subsection constructs Layer L3 (§3.4), which goes beyond the coverage criterion of decision table testing. It has two parts, L3a and L3b.

L3a: boundary combinations. Each L2 test places one condition at or next to its boundary, with two exceptions noted in §4.8: TC-ATM-17 also puts c_3 on T_3, and TC-ATM-11 and TC-ATM-18 fail both parts of R2. The L2 tests of c_5 use the nominal amount, so they place the balance on the threshold of that amount only. The combinations are chosen by one rule: each pairs a boundary of c_2 or c_3 with the balance threshold that this boundary implies, or changes c_4 while all numeric inputs stay on a threshold (Table 10). External variants at the amount boundaries (thresholds 100,000 and 20,050,000) are not included; they would repeat AC1 and AC2 with the fee added.

Table 10: ATM boundary combinations (Layer L3a)

ID	Test	Inputs (bal / used / amt / status / internal)	Why it is needed
AC1	TC-ATM-29	50,000 / 0 / 50,000 / Active / T	c_2 at Min and c_5 at its threshold $T_5 = 50{,}000$ at once; Approved only if both R2 and R5 are non-strict.
AC2	TC-ATM-30	20,000,000 / 0 / 20,000,000 / Active / T	c_2 at Max, c_3 at T_3 and c_5 at T_5 at once (three boundaries).
AC3	TC-ATM-31	500,000 / 19,500,000 / 500,000 / Active / T	The two derived conditions c_3 and c_5 both exactly at their thresholds.
AC4	TC-ATM-32	500,000 / 0 / 500,000 / Active / F	Same numeric inputs as TC-ATM-24 (Approved); only c_4 changes, which moves T_5 from 500,000 to 550,000 and turns the outcome into Rejected_InsufficientBalance.

L3b: don't-care expansion. Each collapsed rejection rule DR1–DR5 contains don't-care entries that encode the precedence order: DR5, for example, asserts that a failing daily limit is reported regardless of the balance. A test of DR5 at its nominal input (TC-ATM-05) has $c_5 = \mathrm{T}$,

so it cannot distinguish R3-before-R5 from R5-before-R3: with only one failing rule, either order reports the same reason. The order becomes observable only when the don't-care entry of the rule evaluated next in the precedence order is set to its failing entry, so that two adjacent rules fail at once. Table 11 applies this to each rejection rule.

Table 11: ATM don't-care expansion (Layer L3b)

ID	Rule	Test	Inputs (bal / used / amt / status / internal)	Rules failing / expected
AX1	DR1	TC-ATM-33	25,000,000 / 0 / 49,999 / Frozen / T	R1 and R2; Rejected_InvalidAccount
AX2	DR2	TC-ATM-34	25,000,000 / 20,000,000 / 49,999 / Active / T	R2 (A_1) and R3; Rejected_InvalidAmount
AX3	DR3	TC-ATM-35	25,000,000 / 20,000,000 / 50,001 / Active / T	R2 (A_3) and R3; Rejected_InvalidAmount
–	DR4	TC-ATM-04, -18, -19	(L1/L2 tests)	R2 (A_4) and R3: $c_3 = \mathrm{F}$ is forced for every A_4 amount (§4.6), so the expansion is already present
AX4	DR5	TC-ATM-36	499,999 / 19,500,001 / 500,000 / Active / T	R3 and R5; Rejected_DailyLimit

For each rejection rule only the next rule in the precedence order is expanded. Expanding every don't-care entry to every value would restore the 56 feasible columns of the full table; the next rule is the one whose failure would be reported if two adjacent rules were evaluated in the opposite order, which is the fault the expansion is meant to expose.

4.10 Test Suite Construction

The suite was built in the four layers of §3.4.

L1 (9 tests, TC-ATM-01–09).
One test per rule DR1–DR9 of Table 6. Each starts from the nominal configuration (Table 4) and changes only what its rule requires (Table 12).

L2 (19 tests, TC-ATM-10–28).
The coverage items of Tables 7–9, except the nominal values already used by L1 tests.

L3 (8 tests, TC-ATM-29–36).
The boundary combinations AC1–AC4 (L3a, Table 10) and the don't-care expansions AX1–AX4 (L3b, Table 11).

L4 (1 test, TC-ATM-37).
The tests needed to kill designed mutants that L1–L3 leave alive. One such mutant exists: M8, which checks divisibility by 10,000 instead of 50,000 (§4.12). Every amount in L1–L3 that is not a multiple of 50,000 is also not a multiple of 10,000 (25,000, 49,999, 50,001, 512,345, 19,999,999, 20,000,001), so M8 rejects all of them for the same reason as the specification. TC-ATM-37 uses 60,000, a multiple of 10,000 but not of 50,000.

No two tests of a layer share their rule, boundary coverage item and combination. Table 13 lists the 37 tests; every expected output was computed from the formal specification of §4.2.

Table 12: Nominal inputs of the L1 tests (changes against Table 4)

Rule	Test	Change	Why this value
DR1	TC-ATM-01	status Frozen	the only invalid status
DR2	TC-ATM-02	amount 25,000	A_1 is unbounded below; 25,000 is the midpoint of its non-negative part $[0, 50{,}000)$, where ordinary requests lie
DR3	TC-ATM-03	amount 512,345	in range, 12,345 away from the nearest multiple of 50,000
DR4	TC-ATM-04	amount 30,000,000	10,000,000 above the range
DR5	TC-ATM-05	dailyUsed 19,750,000	daily sum 20,250,000, the middle of the sums (20,000,000; 20,500,000] reachable with the nominal amount when *dailyUsed* does not itself exceed the daily limit
DR6, DR8	TC-ATM-06, -08	balance 100,000 (DR8: external)	far below both thresholds 500,000 and 550,000
DR7	TC-ATM-07	none	the nominal configuration
DR9	TC-ATM-09	external	balance 25,000,000 far above 550,000

A granularity slip that the suite does not expose. M8 is one of several possible wrong moduli. Replacing 50,000 by a divisor d of it changes R2 only for in-range amounts that are multiples of d but not of 50,000. Every divisor of 10,000 is exposed by TC-ATM-37, but the divisors of $25{,}000 = 50{,}000/2$ that do not divide 10,000, such as 25,000 and 12,500, are exposed by no designed test: no amount of the suite is an odd multiple of 25,000 inside the range. An amount such as 75,000 would be needed. This follows by substitution of the suite's amounts and is confirmed by execution in §4.13; it shows that a mutation-feedback test protects against the mutant it was added for, not against the whole class of similar faults.

Table 13: ATM test suite with traceability (inputs: *balance / dailyUsed / amount / status / isInternal*; Act = Active, Frz = Frozen)

ID	Inputs	Layer	Rule	Boundary / combination	Expected output
TC-ATM-01	25,000,000 / 0 / 500,000 / Frz / T	L1	DR1	–	Rejected_InvalidAccount
TC-ATM-02	25,000,000 / 0 / 25,000 / Act / T	L1	DR2	–	Rejected_InvalidAmount
TC-ATM-03	25,000,000 / 0 / 512,345 / Act / T	L1	DR3	–	Rejected_InvalidAmount
TC-ATM-04	25,000,000 / 0 / 30,000,000 / Act / T	L1	DR4	–	Rejected_InvalidAmount
TC-ATM-05	25,000,000 / 19,750,000 / 500,000 / Act / T	L1	DR5	–	Rejected_DailyLimit
TC-ATM-06	100,000 / 0 / 500,000 / Act / T	L1	DR6	–	Rejected_InsufficientBalance
TC-ATM-07	25,000,000 / 0 / 500,000 / Act / T	L1	DR7	nominal	Approved
TC-ATM-08	100,000 / 0 / 500,000 / Act / F	L1	DR8	–	Rejected_InsufficientBalance

ID	Inputs	Layer	Rule	Boundary / combination	Expected output
TC-ATM-09	25,000,000 / 0 / 500,000 / Act / F	L1	DR9	–	Approved
TC-ATM-10	25,000,000 / 0 / 0 / Act / T	L2	DR2	c_2: Min^- (lattice)	Rejected_InvalidAmount
TC-ATM-11	25,000,000 / 0 / 49,999 / Act / T	L2	DR2	c_2: Min^- (integer)	Rejected_InvalidAmount
TC-ATM-12	25,000,000 / 0 / 50,000 / Act / T	L2	DR7	c_2: Min	Approved
TC-ATM-13	25,000,000 / 0 / 50,001 / Act / T	L2	DR3	c_2: Min^+ (integer)	Rejected_InvalidAmount
TC-ATM-14	25,000,000 / 0 / 100,000 / Act / T	L2	DR7	c_2: Min^+ (lattice)	Approved
TC-ATM-15	25,000,000 / 0 / 19,950,000 / Act / T	L2	DR7	c_2: Max^- (lattice)	Approved
TC-ATM-16	25,000,000 / 0 / 19,999,999 / Act / T	L2	DR3	c_2: Max^- (integer)	Rejected_InvalidAmount
TC-ATM-17	25,000,000 / 0 / 20,000,000 / Act / T	L2	DR7	c_2: Max	Approved
TC-ATM-18	25,000,000 / 0 / 20,000,001 / Act / T	L2	DR4	c_2: Max^+ (integer)	Rejected_InvalidAmount
TC-ATM-19	25,000,000 / 0 / 20,050,000 / Act / T	L2	DR4	c_2: Max^+ (lattice)	Rejected_InvalidAmount
TC-ATM-20	25,000,000 / 19,499,999 / 500,000 / Act / T	L2	DR7	c_3: $T_3 - 1$	Approved
TC-ATM-21	25,000,000 / 19,500,000 / 500,000 / Act / T	L2	DR7	c_3: T_3	Approved
TC-ATM-22	25,000,000 / 19,500,001 / 500,000 / Act / T	L2	DR5	c_3: $T_3 + 1$	Rejected_DailyLimit
TC-ATM-23	499,999 / 0 / 500,000 / Act / T	L2	DR6	c_5 internal: $T_5 - 1$	Rejected_InsufficientBalance
TC-ATM-24	500,000 / 0 / 500,000 / Act / T	L2	DR7	c_5 internal: T_5	Approved
TC-ATM-25	500,001 / 0 / 500,000 / Act / T	L2	DR7	c_5 internal: $T_5 + 1$	Approved
TC-ATM-26	549,999 / 0 / 500,000 / Act / F	L2	DR8	c_5 external: $T_5 - 1$	Rejected_InsufficientBalance
TC-ATM-27	550,000 / 0 / 500,000 / Act / F	L2	DR9	c_5 external: T_5	Approved
TC-ATM-28	550,001 / 0 / 500,000 / Act / F	L2	DR9	c_5 external: $T_5 + 1$	Approved
TC-ATM-29	50,000 / 0 / 50,000 / Act / T	L3a	DR7	AC1	Approved
TC-ATM-30	20,000,000 / 0 / 20,000,000 / Act / T	L3a	DR7	AC2	Approved
TC-ATM-31	500,000 / 19,500,000 / 500,000 / Act / T	L3a	DR7	AC3	Approved
TC-ATM-32	500,000 / 0 / 500,000 / Act / F	L3a	DR8	AC4	Rejected_InsufficientBalance
TC-ATM-33	25,000,000 / 0 / 49,999 / Frz / T	L3b	DR1	AX1 (R1 $\wedge$ R2 fail)	Rejected_InvalidAccount

ID	Inputs	Layer	Rule	Boundary / combination	Expected output
TC-ATM-34	25,000,000 / 20,000,000 / 49,999 / Act / T	L3b	DR2	AX2 (R2 ∧ R3 fail)	Rejected_InvalidAmount
TC-ATM-35	25,000,000 / 20,000,000 / 50,001 / Act / T	L3b	DR3	AX3 (R2 ∧ R3 fail)	Rejected_InvalidAmount
TC-ATM-36	499,999 / 19,500,001 / 500,000 / Act / T	L3b	DR5	AX4 (R3 ∧ R5 fail)	Rejected_DailyLimit
TC-ATM-37	25,000,000 / 0 / 60,000 / Act / T	L4	DR3	multiple of 10,000, not of 50,000	Rejected_InvalidAmount

Coverage check. Table 14 reads the suite in the opposite direction: from each rule of the decision table to the tests that exercise it. Every feasible rule has an L1 test, so the suite achieves 100 % decision table coverage (9/9 feasible collapsed rules) [15, §4.2.3]; L2 tests then reach every rule that has an ordered condition boundary next to it. Every boundary coverage item of Tables 7–9 is exercised by at least one test (some also by the combinations of L3a), and every element of D_{out} is the expected output of at least one test.

Table 14: ATM decision-table rule coverage

Rule	Output	L1 test	Further tests (layer)
DR1	Rejected_InvalidAccount	TC-ATM-01	TC-ATM-33 (L3b)
DR2	Rejected_InvalidAmount	TC-ATM-02	TC-ATM-10, -11 (L2); TC-ATM-34 (L3b)
DR3	Rejected_InvalidAmount	TC-ATM-03	TC-ATM-13, -16 (L2); TC-ATM-35 (L3b); TC-ATM-37 (L4)
DR4	Rejected_InvalidAmount	TC-ATM-04	TC-ATM-18, -19 (L2)
DR5	Rejected_DailyLimit	TC-ATM-05	TC-ATM-22 (L2); TC-ATM-36 (L3b)
DR6	Rejected_InsufficientBalance	TC-ATM-06	TC-ATM-23 (L2)
DR7	Approved	TC-ATM-07	TC-ATM-12, -14, -15, -17, -20, -21, -24, -25 (L2); TC-ATM-29–31 (L3a)
DR8	Rejected_InsufficientBalance	TC-ATM-08	TC-ATM-26 (L2); TC-ATM-32 (L3a)
DR9	Approved	TC-ATM-09	TC-ATM-27, -28 (L2)

4.11 Mutation Model

Table 15 lists fourteen designed mutants. Each corresponds to a plausible implementation slip in one condition, constant, connective or evaluation order of the specification; M11 and M14 reverse the order of two adjacent rules, which the decision table records only as don't-care entries.

Table 15: ATM mutants

ID	Original	Mutated	Fault type	Targeted condition / boundary
M1	$balance \geq amount + fee$ (R5)	$balance > amount + fee$	Relational operator	c_5: threshold T_5
M2	$dailyUsed + amount \leq$ 20,000,000 (R3)	$dailyUsed + amount <$ 20,000,000	Relational operator	c_3: threshold T_3
M3	20,000,000 (upper amount bound, R2)	19,999,999	Boundary constant −1	c_2: Max
M4	20,000,000 (upper amount bound, R2)	20,000,001	Boundary constant +1	c_2: Max – **equivalent mutant** (see discussion below)

ID	Original	Mutated	Fault type	Targeted condition / boundary
M5	Gate requires *both* to hold (R1 $\wedge$ R2)	Gate requires only *either* (R1 $\vee$ R2); R3/R5 unchanged	Logical connective (AND$\rightarrow$OR)	Interaction of status and amount validity (only the R1/R2 gate is mutated; R3, R5 are unaffected)
M6	*fee* = 50,000 if external	*fee* = 0 always (fee removed)	Missing computation	c_4/c_5 interaction
M7	*fee* = 50,000 if external	*fee* = 5,000 if external	Wrong constant	Fee magnitude
M8	$amount \equiv 0 \pmod{50{,}000}$ (R2)	$amount \equiv 0 \pmod{10{,}000}$	Wrong divisibility constant	c_2: granularity (A_2/A_3)
M9	*accountStatus* = Active (R1)	*accountStatus* = Frozen (condition reversed)	Reversed categorical condition	c_1
M10	50,000 (lower amount bound, R2)	0	Boundary constant (lower bound lowered)	c_2: Min
M11	Precedence order R1, R2, R3, R5	Precedence order R1, R2, R5, R3 (R3/R5 swapped)	Control-flow reordering	Precedence R3/R5 (don't-care of DR5)
M12	$dailyUsed + amount \leq$ 20,000,000 (R3)	$dailyUsed \leq$ 20,000,000 (amount dropped from sum)	Arithmetic mutation	c_3: derived sum
M13	*fee* = 50,000 if external	*fee* = 100,000 if external	Wrong constant	Fee magnitude (overcharge direction)
M14	Precedence order R1, R2, R3, R5	Precedence order R2, R1, R3, R5 (R1/R2 swapped)	Control-flow reordering	Precedence R1/R2 (don't-care of DR1)

4.12 Test–Mutant Analysis

Table 16 gives the **analytically expected** result of every (test, mutant) pair. Each cell was obtained by substituting the test's five inputs into the specification of §4.2 and into the mutated rule set, evaluating R1–R5 under the precedence order of each, and comparing the two elements of D_{out}: `K` if they differ, `S` if the mutated rule is reached and the outputs agree, and `-` if an earlier rule already fails identically in both, so that the mutated rule is never reached. For M5 (the R1/R2 gate weakened from AND to OR) the substitution uses the following reading: if exactly one of R1, R2 fails, evaluation continues with R3 and R5 as if both had passed; if both fail, the result is Rejected_InvalidAccount. The matrix is compared with the executed kill sets in §4.13.

Table 16: ATM Test×Mutant matrix (analytically expected and confirmed by execution; `K` = kill, `S` = survive, `-` = mutated rule not reached)

Test	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	M13	M14
TC-ATM-01	–	–	–	–	K	–	–	–	K	–	–	–	–	S
TC-ATM-02	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-03	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-04	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-05	–	S	S	S	S	–	–	S	K	S	S	K	–	S
TC-ATM-06	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-07	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-08	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-09	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-10	–	–	S	S	K	–	–	S	K	K	–	–	–	S
TC-ATM-11	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-12	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-13	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-14	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-15	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-16	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-17	S	K	K	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-18	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-19	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-20	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-21	S	K	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-22	–	S	S	S	S	–	–	S	K	S	S	K	–	S
TC-ATM-23	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-24	K	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-25	S	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-26	S	S	S	S	S	K	K	S	K	S	S	S	S	S
TC-ATM-27	K	S	S	S	S	S	S	S	K	S	S	S	K	S
TC-ATM-28	S	S	S	S	S	S	S	S	K	S	S	S	K	S
TC-ATM-29	K	S	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-30	K	K	K	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-31	K	K	S	S	S	S	S	S	K	S	S	S	S	S
TC-ATM-32	S	S	S	S	S	K	S	S	K	S	S	S	S	S
TC-ATM-33	–	–	–	–	S	–	–	–	K	–	–	–	–	K
TC-ATM-34	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-35	–	–	S	S	K	–	–	S	K	S	–	–	–	S
TC-ATM-36	–	S	S	S	S	–	–	S	K	S	K	K	–	S
TC-ATM-37	–	–	S	S	K	–	–	K	K	S	–	–	–	S
Kills	5	4	2	0	13	2	1	1	37	1	1	3	2	1

Discussion of the columns. The mutants fall into four groups, according to the kind of test that kills them.

Equality-boundary mutants (M1, M2, M3). Each changes the outcome only for inputs that lie exactly on a boundary, so each is killed exactly by the tests that sit on that boundary. M1 (R5 strict) is killed by the five tests with *balance* $= T_5$: TC-ATM-24 and TC-ATM-27 (the T_5 items of the internal and external columns) and the combinations TC-ATM-29–31. M2 (R3 strict) is killed by the four tests with a daily sum of exactly 20,000,000: TC-ATM-17, -21, -30 and -31; TC-ATM-22 and TC-ATM-36, whose sum is 20,000,001, are rejected identically by the original and the mutant. M3 (upper bound 19,999,999) is killed by the two tests with *amount* = 20,000,000, TC-ATM-17 and TC-ATM-30. No L1 test kills any of the three, because every L1 input is a nominal value away from all boundaries.

Threshold-magnitude mutants (M6, M7, M10, M13). These move a threshold by a fixed amount. They are killed only by a test placed between the correct and the mutated threshold. M7 (external fee 5,000, so $T_5 = 505{,}000$) is killed only by TC-ATM-26 (549,999); TC-ATM-32 (500,000) and TC-ATM-08 (100,000) lie below both thresholds. M6 (fee removed, $T_5 = 500{,}000$) is killed by TC-ATM-26 and TC-ATM-32. M13 (fee 100,000, $T_5 = 600{,}000$) is killed by TC-ATM-27 and TC-ATM-28, the external T_5 and $T_5 + 1$ items. M10 (lower amount bound 0) is killed only by TC-ATM-10: the mutant accepts exactly the multiples of 50,000 in $[0, 50{,}000)$,

i.e. the amount 0, and 0 is the only amount below the range in the suite that is a multiple of 50,000; the other amounts below the range (25,000, 49,999) fail the granularity condition and are rejected by the mutant as well. All four are killed by L2 tests derived from the $T_5 - 1$, T_5, $T_5 + 1$ items of the external column and from the lattice item Min^-.

Precedence mutants (M11, M14). They swap two adjacent rules. With at most one failing rule, either order reports the same reason, so each is killed only by a test in which both rules of its pair fail: M11 (R3/R5) by TC-ATM-36 and M14 (R1/R2) by TC-ATM-33, the don't-care expansions AX4 and AX1, which were constructed to make exactly these pairs fail; their kills are therefore by construction. No test of the decision table coverage layer or the boundary layer kills them: the only tests of those layers in which two rules fail are TC-ATM-04, -18 and -19 (an amount above the range with $dailyUsed = 0$, so R2 and R3 fail, and for TC-ATM-04 also R5), and neither M11 nor M14 concerns that pair. The third adjacent reversal, R2/R3, is not among the designed mutants; by substitution it would already be killed by TC-ATM-04, -18 and -19, because the rule DR4 forces $c_3 = \text{F}$ for every amount above the range. The claim that only the don't-care expansion exposes a reversed precedence therefore holds for the two modelled pairs, not for every pair; all six transpositions of two checks are executed in §4.13.

Coarse mutants (M5, M9, M12). These change the outcome for whole regions of a rule, so any test that reaches the mutated rule in such a region kills them. M9 (status condition reversed) is killed by all 37 tests: on the 35 tests with an active account it reports Rejected_InvalidAccount immediately; on TC-ATM-01 it approves a frozen account, and on TC-ATM-33 it passes R1 and reports Rejected_InvalidAmount. M5 (R1/R2 gate weakened to OR) is killed by the 13 tests in which exactly one of R1, R2 fails; on TC-ATM-01, for example, the frozen account proceeds to R3 and R5 and is approved. It survives TC-ATM-33, where both fail and the mutant reports the same reason as the specification. M12 (amount dropped from the daily sum) is killed by the three tests with $dailyUsed \leq 20{,}000{,}000 < dailyUsed + amount$ that reach R3: TC-ATM-05, -22 and -36. These three mutants are the only ones that the L1 tests already kill.

M8 (divisibility by 10,000) is the one mutant that no test of L1–L3 kills; the L4 test TC-ATM-37 (§4.10) exists for this reason and is, by construction, the only test that kills it.

Equivalent mutant. M4 raises the upper amount bound to 20,000,001. Every amount accepted by R2 is a multiple of 50,000, and 20,000,001 is not, so the mutated range admits no additional valid amount: M4 computes the same output as the specification for every input of the domain, not only for the inputs of this suite, and is excluded from the denominator of the mutation score (§2.5). The other thirteen mutants are each killed by at least one test, so the analytically expected score of the suite is 13/13. This is a property of the design against the stated specification; its executed confirmation, and a score measured on an independent mutant population, are reported in §4.13. The designed mutants were written with knowledge of the boundaries the suite targets (Section 7), so the full score is not, by itself, evidence that boundary tests are generally stronger than interior ones.

4.13 Executed Validation

This subsection reports the executed stage of §3.6 for the ATM validator.

Implementation decisions. The specification leaves two points open, so the implementation makes an explicit choice for each and tests it. First, $dailyUsed < 0$, which AS2 excludes from the domain, raises an argument-range error instead of returning an element of D_{out}; this check runs before R1, so a frozen account with a negative *dailyUsed* also raises it. Second, R3 is evaluated in the rearranged form $dailyUsed > 20{,}000{,}000 - amount$. It is equivalent to R3 for every input that reaches it (R2 has already bounded *amount*), but unlike the direct sum it cannot overflow when *dailyUsed* is close to the largest representable integer; the specification does not address overflow.

Suite execution. All 131 ATM tests pass: the 37 designed tests TC-ATM-01–37, 12 supplementary tests (domain guards, overflow extremes, an undefined status value, a negative balance, a negative amount and two further daily-limit values), 5 decision table checks, 65 matrix-reproduction checks (including the four granularity checks of §4.10 and the seven precedence-transposition checks below), 4 layer-ablation checks and 8 properties (Table 2).

Decision table checks. The 56 feasible columns of the full table are executed with one witness input each. Every witness matches exactly one collapsed rule, and the implementation returns that rule's output for all of them; the rules stand for $28+8+8+4+4+4 = 56$ feasible columns, as derived. Witnesses are constructed per column, so this check tests the implementation against the table, not the table itself. The eight columns with $c_2 = A_4$ and $c_3 = \mathrm{T}$ are infeasible by the argument of §4.6 and are not executed. Every designed test belongs to the rule recorded for it in Table 13.

Matrix reproduction. The analytical matrix of Table 16 was derived by substitution, without executing the mutant models, and is compared with the executed kill sets: in all $37 \times 14 = 518$ cells, an analytical `K` coincides with an executed kill and every `S` or `-` with an executed survival. The executed score is therefore 13/13 non-equivalent designed mutants. The equivalence of M4 is additionally checked by executing the original and M4 for every integer amount in [−100,000, 20,200,000] (20,300,001 values, all other rules passing); no output differs. Four further checks confirm the granularity result of §4.10: of the moduli 10,000, 5,000, 25,000 and 12,500, the first two are exposed only by TC-ATM-37 and the last two by no designed test.

Layer ablation. Table 17 reports how many of the 13 non-equivalent designed mutants and of the 22 tool-generated mutants (below) the designed tests of the cumulative layers kill, with L3 split into its two parts, and which designed mutants each layer kills on its own.

Table 17: ATM layer ablation: mutants killed by the designed tests of the cumulative layers, and by each layer alone

Layers	Tests	Designed (of 13)	Stryker (of 22)	Designed mutants killed by the added layer alone
L1	9	3	14	L1: M5, M9, M12
L1–L2	28	10	21	L2: M1–M3, M5–M7, M9, M10, M12, M13
L1–L3a	32	10	21	L3a: M1–M3, M6, M9
L1–L3	36	12	21	L3b: M5, M9, M11, M12, M14
L1–L4	37	13	21	L4: M5, M8, M9
Size-matched baseline: L1 + 19 random interior tests (1,000 draws)				
	28	4.345 (3–8)	–	mean (range) of killed designed mutants

The Stryker column was obtained by running the tool once for L1, L1–L2, L1–L3 and all layers, with the designed suite restricted to those layers through the environment variable `SWT_LAYERS` and the option `disable-bail`, and counting the mutants that at least one designed test kills; the other test groups stay in the run and are not counted. Since L1–L2 already reaches 21, the largest number reachable inside the domain, the L1–L3a value follows without a separate run.

On both populations the decision table layer alone is weak: its nine nominal tests kill 3 of the 13 designed and 14 of the 22 tool-generated mutants. Of the eight tool-generated mutants it misses, seven are four relational mutants, at the amount bounds ($<\to\leq$ at 50,000, $>\to\geq$ at 20,000,000), at the daily limit and at the balance threshold, and three mutants that change the fee or the way it enters the threshold (fee always 0, fee always 50,000, $\mathit{amount} - \mathit{fee}$); all seven

are killed by the boundary layer, and the eighth removes the *dailyUsed* ≥ 0 guard (see below). The boundary combinations (L3a) add no kill on either population: the mutants they kill are already killed by L1–L2. The don't-care expansion (L3b) adds M11 and M14, by construction (§4.12), and L4 adds M8, also by construction. Stryker.NET has no operator that reorders two rules or replaces the modulus by 10,000, so the faults that M11, M14 and M8 represent do not occur in the tool-generated population, and L3b and L4 add nothing there.

The last column of Table 17 shows that L2 alone kills the same ten designed mutants as L1–L2, including the three that L1 kills: on these mutants L1 adds no kill to L2, and its contribution is the set of rules within which the L2 inputs are placed. The L1 row uses the refined table; the L1 tests of the seven rules of the minimal table (DR1–DR7, with DR8 and DR9 merged into DR6 and DR7) kill the same three designed mutants. The L1 result also depends on the nominal values of Table 12: by substitution, a DR8 balance in [505,000; 549,999] instead of 100,000 would kill M6 and M7, and a DR9 balance in [550,000; 599,999] instead of 25,000,000 would kill M13, already in L1. In the same way, the exposure of the transposition R1/R5 by TC-ATM-04 (below) depends on its amount exceeding the nominal balance.

Size-matched baseline. To separate the effect of boundary values from the effect of more tests, the baseline puts 19 inputs drawn at random from the interiors of the same nine rules in the place of the 19 L2 tests: a rule uniformly at random, then each condition entry of the rule satisfied by a uniformly drawn value (amounts in A_1 from $[0, 49{,}999]$, in A_2 as $k \cdot 50{,}000$ with $k \in [1, 400]$, in A_3 from $[50{,}001, 19{,}999{,}999]$, in A_4 from $[20{,}000{,}001, 40{,}000{,}000]$; *dailyUsed* from $[0,\ 20{,}000{,}000 - amount]$ when c_3 must hold and from $[\max(0, 20{,}000{,}001 - amount),\ 20{,}000{,}000]$ when it must fail; *balance* from $[T_5,\ T_5 + 10{,}000{,}000]$ or $[T_5 - 10{,}000{,}000,\ T_5 - 1]$; don't-care entries drawn at random, with *dailyUsed* from $[0, 20{,}000{,}000]$ and *balance* from $[0, 30{,}000{,}000]$). Over 1,000 draws with the seed stated in §3.6, L1 plus the 19 random tests kill on average 4.345 designed mutants (minimum 3, maximum 8), against 10 for L1–L2. M1, M8 and M10 are never killed; M2, M3, M6, M7 and M13, which need an input on or between thresholds, are killed in 1.7 %, 2.0 %, 0.6 %, 0.5 % and 1.2 % of the draws. M11 and M14 are killed in 50.0 % and 78.5 % of the draws, because a random member of DR5 or DR1 often fills a don't-care entry with a failing value, which is a don't-care expansion by chance. Compared mutant by mutant, the random tests add on average 0.06 kills on the seven mutants that L2 adds (M1–M3, M6, M7, M10, M13), where L2 adds 7, and 1.285 kills on M11 and M14, which L2 does not kill. The distributions are a property of the stated sampling scheme; a different scheme gives different numbers.

Other precedence transpositions. M14 and M11 are two of the six transpositions of a pair of checks in the order R1, R2, R3, R5. The other four were executed against the designed suite as additional mutants, with the predicates of the specification unchanged and only the order of two checks exchanged. Three of them, R1/R3, R2/R3 and R2/R5, evaluate R3 before R2, and every input of DR4 fails R3, so every test of DR4 exposes them, starting with its L1 test TC-ATM-04. R1/R5 is exposed by TC-ATM-04 only because its amount of 30,000,000 exceeds the nominal balance; with an amount of DR4 at or below the balance it would be exposed only by the don't-care expansions TC-ATM-33 and TC-ATM-36. The tests that expose each transposition are, for R1/R3, TC-ATM-04, -18, -19, -33, -34, -35; for R1/R5, TC-ATM-04, -33, -36; for R2/R3, TC-ATM-04, -18, -19, -34, -35; and for R2/R5, TC-ATM-04, -18, -19, -34, -35, -36. The transpositions R1/R2 and R3/R5 are exposed only by TC-ATM-33 and TC-ATM-36, the kill sets of M14 and M11 in Table 16.

Tool-generated mutants. Stryker.NET generated 23 mutants; one was ignored by the tool as a redundant block removal, no mutant failed to compile, and all other 22 were killed (0 survived). By operator, the killed mutants are equality 10, conditional 4, arithmetic 3, logical 2, negation 2

and statement removal 1. This 100 % score is the score of the complete test project; the designed suite alone kills 21 of the 22 (95 %). The one it does not kill removes the *dailyUsed* ≥ 0 guard; that input lies outside the domain (AS2), so no designed test can reach it, and it is killed by the supplementary guard tests.

4.14 Case Study Discussion

Three observations follow from this case study; the comparison with the second case study is made in §6.1.

First, the decision table made the evaluation order of the specification explicit. The nine collapsed rules close the rule count, and their don't-care entries mark exactly the conditions whose value the precedence R1 → R2 → R3 → R5 makes irrelevant. The coverage criterion of the table is met by nine tests. They cannot observe a reversal of R3/R5 or R1/R2, which the two designed precedence mutants M11 and M14 model; those are killed only by the don't-care expansions AX4 and AX1 (Table 17). The other four transpositions of two checks, executed separately, are already exposed by the coverage test of DR4, TC-ATM-04, whose input fails R2, R3 and R5 together; for three of them this holds for every input of DR4, for R1/R5 only because the amount of TC-ATM-04 exceeds the balance (§4.13).

Second, the decision table says which combinations must be tested but not where inside a rule to test them. Its nine nominal tests kill only the three coarse designed mutants (M5, M9, M12). The boundary values of §4.8 kill seven more; on these seven, nineteen random members of the same rules kill on average 0.06 (§4.13). Four of the seven depend on two details that only the boundary analysis provides: the lattice neighbour $\mathrm{Min}^- = 0$ (M10), and the analysis of c_5 in the external column, where the fee moves the threshold to 550,000 (M6, M7, M13). The boundary combinations of L3a, in contrast, kill nothing that L1–L2 do not already kill. The granularity condition remains the weakest point of the suite: a modulus of 25,000 survives every test.

Third, a categorical input can change a numeric boundary. c_4 (*isInternal*) never decides an outcome by itself, yet it cannot be collapsed, because it selects the fee and with it the threshold of c_5 (Figure 3); the combination AC4 shows the same numeric input approved in one column and rejected in the other. Independent coverage of *isInternal* and of *balance* would not place a test between the two thresholds.

5 Case Study II: Quadratic Equation Solver

5.1 Problem Context

Given integer coefficients a, b, c, classify the number and nature of the real solutions of $ax^2 + bx + c = 0$. Unlike the ATM case, the three inputs here are not coupled by separate policy rules but by a single algebraic identity – the discriminant – whose sign alone determines the qualitative output once the leading-coefficient degeneracy ($a = 0$) has been handled.

5.2 Formal Specification

$$\mathit{Solve}(a, b, c) \to D_{out}, \qquad a, b, c \in \mathbb{Z}.$$

Case A — $a \neq 0$ **(quadratic).** Define $\Delta = b^2 - 4ac$.

$$\Delta < 0 \Rightarrow \text{NoRealRoots} \tag{11}$$

$$\Delta = 0 \Rightarrow \text{OneRepeatedRealRoot} \tag{12}$$

$$\Delta > 0 \Rightarrow \text{TwoDistinctRealRoots} \tag{13}$$

Case B — $a = 0$ (degenerates to linear $bx + c = 0$).

$$b \neq 0 \Rightarrow \text{OneLinearRoot} \tag{14}$$
$$b = 0,\ c \neq 0 \Rightarrow \text{NoSolution} \tag{15}$$
$$b = 0,\ c = 0 \Rightarrow \text{InfinitelyManySolutions} \tag{16}$$

Output domain.

$$D_{out} = \{\text{NoRealRoots},\ \text{OneRepeatedRealRoot},\ \text{TwoDistinctRealRoots},\ \text{OneLinearRoot},\ \text{NoSolution},\ \text{InfinitelyManySolutions}\}$$

Assumptions of this report.

(QS1) Classification only.
The solver reports the real-root classification, not the roots; "no real roots" covers the whole region $\Delta < 0$, since no requirement to report complex roots is given.

(QS2) Test range.
The specification places no bound on a, b, c. The implementation accepts $a, b, c \in [-100, 100]$ and rejects other values with an argument-range error, and the exhaustive sweep of §5.13 runs over the cube $D = [-100, 100]^3 \subset \mathbb{Z}^3$. This range is a property of the test environment, not a behavioural rule: no output changes at ± 100, so it contributes no condition and no boundary to the design (§5.8).

5.3 Input and Output Domains

Table 18: Quadratic solver domains

Input	Domain	Note
a	$\mathbb{Z}$ (tested within $[-100, 100]$)	Leading coefficient; $a = 0$ is the case-split boundary
b	$\mathbb{Z}$ (tested within $[-100, 100]$)	Linear coefficient
c	$\mathbb{Z}$ (tested within $[-100, 100]$)	Constant term

The six output labels are mutually exclusive and, together with Cases A and B, collectively exhaustive: every $(a, b, c) \in \mathbb{Z}^3$ satisfies exactly one of the six conditions $a \neq 0 \wedge \Delta < 0$, $a \neq 0 \wedge \Delta = 0$, $a \neq 0 \wedge \Delta > 0$, $a = 0 \wedge b \neq 0$, $a = 0 \wedge b = 0 \wedge c \neq 0$ and $a = 0 \wedge b = 0 \wedge c = 0$.

5.4 Constraints and Dependencies

a, b, c are not independently meaningful to the output: when $a \neq 0$ the output depends only on the derived quantity $\Delta = b^2 - 4ac$, and when $a = 0$ on a nested case split on b and c. This is analogous to the ATM, whose output depends on the derived quantities $amount + fee$ and $dailyUsed + amount$ rather than on each input alone.

5.5 Nominal Configurations

A single nominal tuple cannot represent the quadratic and the linear behaviour (they are separated by $a = 0$), nor the three signs of Δ. A nominal input is therefore chosen per rule of the decision table (§5.6), and the rules without an interior are identified explicitly (§2.3). Table 19 lists them. A nominal input lies away from the boundary of the condition that defines its rule: Δ is far from 0, and in Case B the non-zero coefficient is not ± 1. In Case A small coefficients keep the arithmetic readable, so two entries are neighbours of an equality boundary: $a = 1$

(QR4–QR7, QR10) is a neighbour of $a = 0$, and $b = 1$ in QR4 is a neighbour of $b = 0$. Those boundaries are analysed separately (§5.8), and the effect of the nominal choice on the mutation results is reported in §5.12.

Table 19: Nominal inputs per decision rule

Rule	(a, b, c)	Justification
QR1	$(0, 2, -4)$	$a = 0$ is required by the rule; $b = 2$ is away from the boundary $b = 0$ that leads to QR2/QR3
QR2	$(0, 0, 5)$	$a = b = 0$ required; $c = 5$ away from $c = 0$ (QR3)
QR3	$(0, 0, 0)$	none (no interior): the rule contains this single triple
QR4	$(1, 1, 10)$	$\Delta = 1 - 40 = -39$, far below 0
QR5	$(1, 2, 1)$	none (no interior): the rule is defined by the equality $\Delta = 0$, so every member lies on the boundary; $(1, 2, 1)$ is the canonical member
QR6	$(1, 7, 6)$	$\Delta = 49 - 24 = 25$, far above 0; $x^2 + 7x + 6 = (x + 1)(x + 6)$
QR7	$(1, 5, 0)$	$c = 0$ required; $\Delta = b^2 = 25$, b away from 0
QR8	$(2, 0, 3)$	$b = 0$ required; $\Delta = -4ac = -24$, far below 0
QR9	$(-2, 0, 3)$	$b = 0$ required; $\Delta = -4ac = 24$, far above 0
QR10	$(1, 0, 0)$	none (no interior): $b = c = 0$ forces $\Delta = 0$ for every $a \neq 0$, so every member $(k, 0, 0)$ lies on the same boundary; $k = 1$ is used

These ten inputs are the decision table coverage tests TC-QE-01 to TC-QE-10 (§5.10). The three rules without an interior are exercised by their canonical members; their boundary character is analysed in §5.8.

5.6 Decision Table

Conditions. The specification evaluates four predicates (Table 20): q_1 ($a = 0$) selects the case; in Case B, q_2 ($b = 0$) and q_3 ($c = 0$) select the output; in Case A, q_4, the sign of Δ, selects the output. q_4 is an extended-entry condition whose three entries $\Delta < 0$, $\Delta = 0$, $\Delta > 0$ are mutually exclusive and exhaust $\mathbb{Z}$ by trichotomy. It is evaluated only in Case A; for the rules of Case B its entry is written "n.e." (not defined), which is different from the ISTQB entry N/A, used there for an infeasible condition [15, §4.2.3].

Table 20: Quadratic decision table: conditions and actions

Stub	Definition	Entries
q_1	$a = 0$	T, F
q_2	$b = 0$	T, F
q_3	$c = 0$	T, F
q_4	sign of $\Delta = b^2 - 4ac$	< 0, $= 0$, > 0 if $q_1 =$ F; n.e. if $q_1 =$ T
a_1	output	one element of D_{out}

Minimal table. Read directly from the specification, the table has six rules: the three rules of Case B ($b \neq 0$; $b = 0, c \neq 0$; $b = c = 0$) and one rule per sign of Δ in Case A, in which q_2 and q_3 are don't-care: they are defined there, but the output of Case A depends only on the sign of Δ.

Refined table (extension). This report refines Case A by the two coefficient conditions q_2 and q_3 that the specification evaluates in Case B. The refinement is an extension of the minimal table and is labelled as such: it does not change any output, but it makes the dependence between q_2, q_3 and q_4 explicit, and it gives the boundaries $b = 0$ and $c = 0$ rules inside Case A from which

their neighbours can be taken (§5.8). With the refinement, Case A has $2 \cdot 2 \cdot 3 = 12$ columns, and five of them are infeasible:

- $b \neq 0,\ c = 0$: $\Delta = b^2 > 0$, so $\Delta < 0$ and $\Delta = 0$ are infeasible (2 columns);
- $b = 0,\ c \neq 0$: $\Delta = -4ac \neq 0$ because $a \neq 0$ and $c \neq 0$, so $\Delta = 0$ is infeasible (1 column);
- $b = 0,\ c = 0$: $\Delta = 0$, so $\Delta < 0$ and $\Delta > 0$ are infeasible (2 columns).

The remaining seven columns are feasible; each has a witness in Table 21. For $b \neq 0, c \neq 0$ all three signs occur ($(1,1,1)$, $(1,2,1)$, $(1,3,2)$); for $b = 0, c \neq 0$ the sign of $\Delta = -4ac$ is the opposite of the sign of ac. Case B has the four columns of (q_2, q_3); the outcome does not depend on c once $b \neq 0$, so two of them collapse into QR1.

Rule count. The full table has $4 + 12 = 16$ columns, of which 5 are infeasible. The ten rules stand for $2 + 1 + 1$ columns in Case B and 7 in Case A, i.e. $2 + 1 + 1 + 7 = 11 = 16 - 5$ feasible columns. This equality is a necessary condition for completeness without overlap; non-overlap follows from the rules having pairwise different entries in at least one condition, and both properties are confirmed by execution over the whole test range (§5.13).

Table 21: Quadratic refined decision table ("−" = don't care; n.e. = not defined)

	QR1	QR2	QR3	QR4	QR5	QR6	QR7	QR8	QR9	QR10
q_1: $a = 0$	T	T	T	F	F	F	F	F	F	F
q_2: $b = 0$	F	T	T	F	F	F	F	T	T	T
q_3: $c = 0$	−	F	T	F	F	F	T	F	F	T
q_4: Δ	n.e.	n.e.	n.e.	< 0	$= 0$	> 0	> 0	< 0	> 0	$= 0$
output	Lin	NoSol	Inf	NoReal	Rep	Two	Two	NoReal	Two	Rep
witness	$(0, 2, -4)$	$(0, 0, 5)$	$(0, 0, 0)$	$(1, 1, 10)$	$(1, 2, 1)$	$(1, 7, 6)$	$(1, 5, 0)$	$(2, 0, 3)$	$(-2, 0, 3)$	$(1, 0, 0)$

Lin = OneLinearRoot, NoSol = NoSolution, Inf = InfinitelyManySolutions, NoReal = NoRealRoots, Rep = OneRepeatedRealRoot, Two = TwoDistinctRealRoots. The minimal table merges QR4 with QR8, QR5 with QR10, and QR6 with QR7 and QR9.

In the refined table, QR7 and QR10 are the two rules in which a zero coefficient fixes the sign of Δ by itself: in QR7 a zero constant term gives $x(ax + b) = 0$ with $b \neq 0$, hence two distinct real roots, and in QR10 a zero linear and constant term give $ax^2 = 0$, hence the repeated root $x = 0$. In QR8 and QR9 the sign of $\Delta = -4ac$ still depends on a and c.

Every input is valid. There is no rejection action: every $(a, b, c) \in \mathbb{Z}^3$ is a legitimate query, and the rules partition the input space rather than filter it.

5.7 Condition Regions

Two pictures support the table. The first shows that q_1 is decided at a single point of the a axis; the second shows the boundary of q_4 in one plane of the input space, which is where the boundary tests of §5.8 are placed. The test marked in the second figure is defined in §5.9.

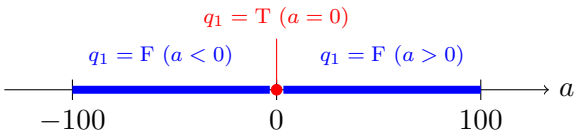


Figure 4: Entries of q_1 on the a axis. $a = 0$ is a single point that does not separate two numeric ranges with the same algorithm but switches between two classification algorithms (quadratic and linear).

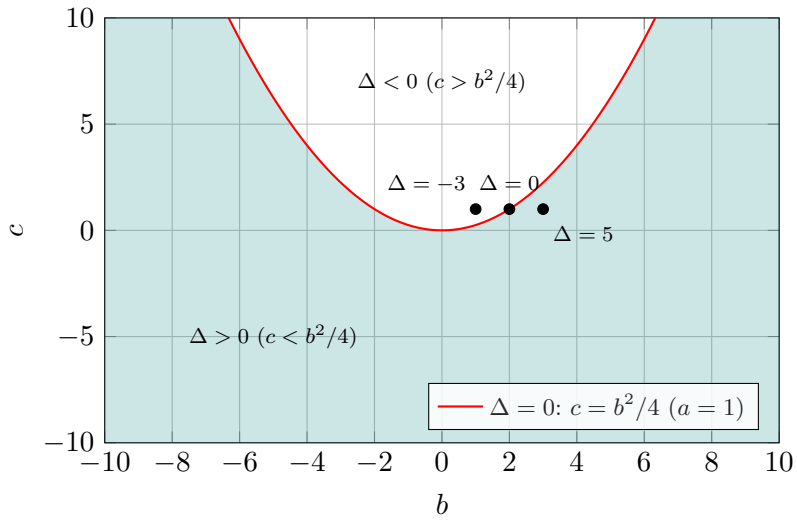


Figure 5: The plane $a = 1$ of the input space, with the curve $b^2 - 4c = 0$ on which $\Delta = 0$. Above the curve $\Delta < 0$, below it (shaded) $\Delta > 0$. The marked points $(b, c) = (1, 1), (2, 1), (3, 1)$, with $\Delta = -3, 0, 5$, are the crossing sequence TC-QE-28 (§5.9). This is one cross-section of the surface $b^2 - 4ac = 0$; its shape, and the side on which $\Delta > 0$ lies, change with the sign and magnitude of a.

5.8 Boundary Value Analysis

Equality conditions q_1, q_2, q_3. Each of $a = 0$, $b = 0$, $c = 0$ partitions an integer coefficient into the ordered classes $\{x < 0\}$, $\{0\}$, $\{x > 0\}$, so its boundary value is 0 and its 3-value BVA coverage items are -1, 0, $+1$. The item 0 is exercised by the L1 test of every rule whose entry for the condition is T. The neighbours ± 1 are taken from each such rule, starting from its nominal input and changing only the coefficient concerned, whenever the change alters the output. Neighbours that do not alter the output lie in a rule with the same action and are not used as boundary tests; this is the case next to QR7 $(1, 5, 0)$, where $c = \pm 1$ gives $\Delta = 25 \mp 4 > 0$, next to QR8 $(2, 0, 3)$, where $b = \pm 1$ gives $\Delta = 1 - 24 < 0$, and next to QR9 $(-2, 0, 3)$, where $b = \pm 1$ gives $\Delta = 1 + 24 > 0$. Table 22 lists the neighbours used.

Table 22: 3-value BVA of the equality conditions: neighbours that change the output

Condition	From rule	Neighbour	Test	Expected / rule reached
q_1: $a = 0$	QR1 $(0, 2, -4)$	$(-1, 2, -4)$	TC-QE-11	NoRealRoots (QR4, $\Delta = -12$)
		$(1, 2, -4)$	TC-QE-12	TwoDistinctRealRoots (QR6, $\Delta = 20$)
	QR2 $(0, 0, 5)$	$(-1, 0, 5)$	TC-QE-13	TwoDistinctRealRoots (QR9, $\Delta = 20$)
		$(1, 0, 5)$	TC-QE-14	NoRealRoots (QR8, $\Delta = -20$)
	QR3 $(0, 0, 0)$	$(-1, 0, 0)$	TC-QE-15	OneRepeatedRealRoot (QR10, $\Delta = 0$)
		$(1, 0, 0)$	TC-QE-10	already the L1 test of QR10
q_2: $b = 0$, $a = 0$	QR2 $(0, 0, 5)$	$(0, -1, 5)$, $(0, 1, 5)$	TC-QE-16, -17	OneLinearRoot (QR1)
	QR3 $(0, 0, 0)$	$(0, -1, 0)$, $(0, 1, 0)$	TC-QE-18, -19	OneLinearRoot (QR1)
q_3: $c = 0$, $a = b = 0$	QR3 $(0, 0, 0)$	$(0, 0, -1)$, $(0, 0, 1)$	TC-QE-20, -21	NoSolution (QR2)
q_2: $b = 0$, $a \neq 0$	QR10 $(1, 0, 0)$	$(1, -1, 0)$, $(1, 1, 0)$	TC-QE-22, -23	TwoDistinctRealRoots (QR7, $\Delta = 1$)
q_3: $c = 0$, $a \neq 0$	QR10 $(1, 0, 0)$	$(1, 0, -1)$	TC-QE-24	TwoDistinctRealRoots (QR9, $\Delta = 4$)
		$(1, 0, 1)$	TC-QE-25	NoRealRoots (QR8, $\Delta = -4$)

Sign of the discriminant (q_4): reachable neighbours of $\Delta = 0$. The boundary of q_4 is $\Delta = 0$, a joint condition on (a, b, c), not a boundary of any one coefficient: the surface $b^2 - 4ac = 0$ is two-dimensional inside the three-dimensional input space, and it can be crossed by changing any one coefficient while the other two are fixed. Its integer neighbours $\Delta = \pm 1$ are not both reachable. For any integer b, $b^2 \equiv 0$ or $1 \pmod 4$, and $4ac \equiv 0 \pmod 4$, so

$$\Delta = b^2 - 4ac \equiv 0 \text{ or } 1 \pmod 4.$$

Hence $\Delta = -1$ and $\Delta = -2$ are unreachable, and the nearest reachable values below and above 0 are -3 and $+1$ (Table 23). In the rules with $b = 0$, $\Delta = -4ac$ is a non-zero multiple of 4, so there the nearest reachable values are ± 4, already exercised by TC-QE-24 and TC-QE-25.

Table 23: Reachable discriminant values next to $\Delta = 0$ and the tests that use them

Item	Witness (a, b, c)	Test	Check
nearest reachable $\Delta < 0$: -3	$(1, 1, 1)$	TC-QE-26	$1 - 4 = -3$; NoRealRoots
$\Delta = 0$	$(1, 2, 1)$	TC-QE-05	$4 - 4 = 0$; OneRepeatedRealRoot
nearest reachable $\Delta > 0$: $+1$	$(1, 3, 2)$	TC-QE-27	$9 - 8 = 1$; TwoDistinctRealRoots
± 4 (rules with $b = 0$)	$(1, 0, 1)$, $(1, 0, -1)$	TC-QE-25, -24	∓ 4

No coefficient-range boundary. The specification places no bound on a, b, c (QS2), so no output changes at any coefficient value other than 0, and there is no domain boundary to analyse, as for the balance of the ATM. The extremes ± 100 of the test range and the out-of-range values ± 101 concern the implementation's input check and its arithmetic, and are exercised by supplementary tests (§5.13), not by the designed suite.

5.9 Boundary Combinations and Don't-Care Expansion

Layer L3 (§3.4) adds three tests that the decision table coverage criterion does not require (Table 24). QC3 places two conditions at or next to their boundaries; QC1 and QC2 are of the third kind of L3 test, further points on the derived boundary $\Delta = 0$ reached through other combinations of inputs.

Table 24: Quadratic layer L3: further points on $\Delta = 0$ (QC1, QC2) and a boundary combination (QC3)

ID	Test	(a, b, c)	Why it is needed
QC1	TC-QE-28	$(1,1,1) \to (1,2,1) \to (1,3,1)$	With $a = 1$ and $c = 1$ fixed, b alone is increased by one twice; Δ goes $-3 \to 0 \to 5$ and the output NoRealRoots $\to$ OneRepeatedRealRoot $\to$ TwoDistinctRealRoots. A change of a single coefficient crosses the joint boundary $\Delta = 0$ in both directions (Figure 5). The first two inputs coincide with TC-QE-26 and TC-QE-05, so $(1,3,1)$ is the only new input; the test is the sequence.
QC2	TC-QE-29	$(2,4,2)$	$\Delta = 16 - 16 = 0$ reached by doubling all three coefficients of QR5's $(1,2,1)$: a second point of the surface $\Delta = 0$, with $\lvert a\rvert \neq 1$.
QC3	TC-QE-30	$(-1,2,-1)$	$\Delta = 4 - 4 = 0$ with $a = -1$, a neighbour of the case-split boundary $a = 0$: q_1 next to its boundary and q_4 on its boundary at once.

Don't-care expansion. The only don't-care entry of the refined table is c in QR1, which the L1 test fixes to $c = -4$. Its other entry, $c = 0$, is already instantiated by the L2 tests TC-QE-18 and TC-QE-19 $(0, \pm 1, 0)$, so the expansion needs no further test. Unlike the ATM, the quadratic specification has no evaluation order between failing rules, so there is no precedence for an expansion to expose.

Single coefficients do not reach $\Delta = 0$. For fixed $a \neq 0$ and c, $\Delta = 0$ holds only for $b = \pm\sqrt{4ac}$, which is an integer only when $4ac$ is a perfect square. A value of b chosen without reference to a and c therefore generally misses the boundary, and a boundary test of q_4 has to be constructed by solving $b^2 = 4ac$ for one coefficient given the other two, as QR5, QC1, QC2 and QC3 do. The exhaustive count in §5.13 quantifies how rarely an unconstructed input lands on this boundary.

5.10 Test Suite Construction

The four layers of §3.4 give the following tests.

L1 (10 tests, TC-QE-01–10).
One test per rule QR1–QR10 of the refined table, with the nominal inputs of Table 19. Coverage of the minimal six-rule table needs six of them.

L2 (17 tests, TC-QE-11–27).
The neighbours of the equality conditions that change the output (Table 22) and the nearest reachable discriminants not already used (Table 23).

L3 (3 tests, TC-QE-28–30).
QC1–QC3 (Table 24).

L4 (no test).
Every designed mutant is already killed by layers L1–L3 (§5.12), so mutation feedback adds no test.

Table 25 lists the 30 test cases; TC-QE-28 is one test case that evaluates a sequence of three inputs.

Table 25: Quadratic test suite with traceability (outputs abbreviated as in Table 21)

ID	(a, b, c)	Layer	Rule	Δ	Boundary / combination	Expected
TC-QE-01	$(0, 2, -4)$	L1	QR1	–	nominal	Lin
TC-QE-02	$(0, 0, 5)$	L1	QR2	–	nominal	NoSol
TC-QE-03	$(0, 0, 0)$	L1	QR3	–	single member	Inf
TC-QE-04	$(1, 1, 10)$	L1	QR4	-39	nominal	NoReal
TC-QE-05	$(1, 2, 1)$	L1	QR5	0	canonical ($\Delta = 0$)	Rep
TC-QE-06	$(1, 7, 6)$	L1	QR6	25	nominal	Two
TC-QE-07	$(1, 5, 0)$	L1	QR7	25	nominal	Two
TC-QE-08	$(2, 0, 3)$	L1	QR8	-24	nominal	NoReal
TC-QE-09	$(-2, 0, 3)$	L1	QR9	24	nominal	Two
TC-QE-10	$(1, 0, 0)$	L1	QR10	0	canonical ($b = c = 0$)	Rep
TC-QE-11	$(-1, 2, -4)$	L2	QR4	-12	q_1 from QR1: $a = -1$	NoReal
TC-QE-12	$(1, 2, -4)$	L2	QR6	20	q_1 from QR1: $a = +1$	Two
TC-QE-13	$(-1, 0, 5)$	L2	QR9	20	q_1 from QR2: $a = -1$	Two
TC-QE-14	$(1, 0, 5)$	L2	QR8	-20	q_1 from QR2: $a = +1$	NoReal
TC-QE-15	$(-1, 0, 0)$	L2	QR10	0	q_1 from QR3: $a = -1$	Rep
TC-QE-16	$(0, -1, 5)$	L2	QR1	–	q_2 from QR2: $b = -1$	Lin
TC-QE-17	$(0, 1, 5)$	L2	QR1	–	q_2 from QR2: $b = +1$	Lin
TC-QE-18	$(0, -1, 0)$	L2	QR1	–	q_2 from QR3: $b = -1$	Lin
TC-QE-19	$(0, 1, 0)$	L2	QR1	–	q_2 from QR3: $b = +1$	Lin
TC-QE-20	$(0, 0, -1)$	L2	QR2	–	q_3 from QR3: $c = -1$	NoSol
TC-QE-21	$(0, 0, 1)$	L2	QR2	–	q_3 from QR3: $c = +1$	NoSol
TC-QE-22	$(1, -1, 0)$	L2	QR7	1	q_2 from QR10: $b = -1$	Two
TC-QE-23	$(1, 1, 0)$	L2	QR7	1	q_2 from QR10: $b = +1$	Two
TC-QE-24	$(1, 0, -1)$	L2	QR9	4	q_3 from QR10: $c = -1$	Two
TC-QE-25	$(1, 0, 1)$	L2	QR8	-4	q_3 from QR10: $c = +1$	NoReal

ID	(a, b, c)	Layer	Rule	Δ	Boundary / combination	Expected
TC-QE-26	$(1, 1, 1)$	L2	QR4	-3	q_4: nearest reachable $\Delta < 0$	NoReal
TC-QE-27	$(1, 3, 2)$	L2	QR6	1	q_4: nearest reachable $\Delta > 0$	Two
TC-QE-28	$(1, 1, 1) \to (1, 2, 1) \to (1, 3, 1)$	L3	QR4 $\to$ QR5 $\to$ QR6	$-3 \to 0 \to 5$	QC1 (crossing by b)	NoReal $\to$ Rep $\to$ Two
TC-QE-29	$(2, 4, 2)$	L3	QR5	0	QC2	Rep
TC-QE-30	$(-1, 2, -1)$	L3	QR5	0	QC3 ($a = -1$)	Rep

Coverage check. Table 26 maps each rule to its tests. All ten rules of the refined table have an L1 test, so the suite covers every feasible column of both the refined and the minimal table; every neighbour listed in Tables 22 and 23 is exercised by at least one test, and each of the six outputs is the expected output of at least one test.

Table 26: Quadratic decision-table rule coverage

Rule	Output	L1 test	Further tests (layer)
QR1	OneLinearRoot	TC-QE-01	TC-QE-16–19 (L2)
QR2	NoSolution	TC-QE-02	TC-QE-20, -21 (L2)
QR3	InfinitelyManySolutions	TC-QE-03	–
QR4	NoRealRoots	TC-QE-04	TC-QE-11, -26 (L2); TC-QE-28 (L3, first step)
QR5	OneRepeatedRealRoot	TC-QE-05	TC-QE-28 (L3, second step), -29, -30 (L3)
QR6	TwoDistinctRealRoots	TC-QE-06	TC-QE-12, -27 (L2); TC-QE-28 (L3, third step)
QR7	TwoDistinctRealRoots	TC-QE-07	TC-QE-22, -23 (L2)
QR8	NoRealRoots	TC-QE-08	TC-QE-14, -25 (L2)
QR9	TwoDistinctRealRoots	TC-QE-09	TC-QE-13, -24 (L2)
QR10	OneRepeatedRealRoot	TC-QE-10	TC-QE-15 (L2)

5.11 Mutation Model

Table 27 lists ten designed mutants: three alter the discriminant formula (N1, N6, N10), three alter its sign classification (N2, N3, N5), one reverses the case split on a (N4), and three change the output of one sub-case of the linear branch (N7–N9).

Table 27: Quadratic solver mutants

ID	Original	Mutated	Fault type	Targeted condition / boundary
N1	$\Delta = b^2 - 4ac$	$\Delta = b^2 - 2ac$	Arithmetic (wrong constant)	Discriminant formula, all Δ classes
N2	$\Delta > 0 \Rightarrow$ Two roots	$\Delta \geq 0 \Rightarrow$ Two roots	Relational operator	$\Delta = 0$ boundary
N3	$\Delta < 0 \Rightarrow$ No roots	$\Delta \leq 0 \Rightarrow$ No roots	Relational operator	$\Delta = 0$ boundary
N4	$a = 0 \Rightarrow$ linear case	$a \neq 0 \Rightarrow$ linear case (condition reversed)	Negated case-split condition	$a = 0$ boundary
N5	$\Delta < 0 \Rightarrow$ NoRealRoots, $\Delta > 0 \Rightarrow$ TwoDistinctRealRoots	labels swapped ($\Delta < 0 \Rightarrow$Two, $\Delta > 0 \Rightarrow$No)	Classification swap	Both Δ sign classes
N6	b^2	$2b$	Arithmetic (wrong expression)	Discriminant formula
N7	$b = 0, c = 0 \Rightarrow$ InfinitelyManySolutions	$b = 0, c = 0 \Rightarrow$ NoSolution	Wrong classification	Degenerate linear boundary
N8	$b = 0, c \neq 0 \Rightarrow$ NoSolution	$b = 0, c \neq 0 \Rightarrow$ InfinitelyManySolutions	Wrong classification	$b = 0$ nested boundary
N9	$b \neq 0 \Rightarrow$ OneLinearRoot (under $a = 0$)	$b \neq 0 \Rightarrow$ NoSolution	Wrong classification	$a = 0, b \neq 0$ boundary
N10	$4ac$	$4bc$	Arithmetic (wrong variable)	Discriminant formula

None of the ten mutants replaces one of the zero tests $a = 0$, $b = 0$, $c = 0$ by a relational comparison such as $b \leq 0$: N4 negates the case split, and N7–N9 change an output, not a comparison. The boundary tests of q_2 and q_3 (§5.8) are aimed at such slips, so the mutants of this table cannot measure their contribution; the slips are examined separately in §5.13.

5.12 Test–Mutant Analysis

Of the 300 cells of Table 28, 298 were obtained by substituting the test's coefficients into the specification and into the mutant, computing each one's branch and discriminant, and comparing the two resulting elements of D_{out} – never the raw discriminants. A mutant that changes the value of Δ without moving it across 0 (e.g. $\Delta = 25 \rightarrow 37$) is recorded as `S`, even though the number changed. For N2 and N3 the sign is classified in the order "$\Delta < 0$, then $\Delta > 0$, otherwise $\Delta = 0$", the order in which both relational mutants affect exactly the inputs with $\Delta = 0$. The remaining two cells, N1 at TC-QE-05 and TC-QE-28, show the executed values; §5.13 gives the reason for the difference.

Table 28: Quadratic Test×Mutant matrix (analytically derived; the cells N1 at TC-QE-05 and TC-QE-28 show executed values; `K` = kill, `S` = survive, `-` = mutated branch not reached)

Test	N1	N2	N3	N4	N5	N6	N7	N8	N9	N10
TC-QE-01	–	–	–	K	–	–	–	–	K	–
TC-QE-02	–	–	–	K	–	–	–	K	–	–
TC-QE-03	–	–	–	K	–	–	K	–	–	–
TC-QE-04	S	S	S	K	K	S	–	–	–	S
TC-QE-05	K	K	K	K	S	S	–	–	–	K
TC-QE-06	S	S	S	K	K	K	–	–	–	K
TC-QE-07	S	S	S	K	K	S	–	–	–	S
TC-QE-08	S	S	S	K	K	S	–	–	–	K
TC-QE-09	S	S	S	K	K	S	–	–	–	K
TC-QE-10	S	K	K	K	S	S	–	–	–	S
TC-QE-11	S	S	S	K	K	S	–	–	–	K
TC-QE-12	S	S	S	K	K	S	–	–	–	S
TC-QE-13	S	S	S	K	K	S	–	–	–	K
TC-QE-14	S	S	S	K	K	S	–	–	–	K
TC-QE-15	S	K	K	K	S	S	–	–	–	S
TC-QE-16	–	–	–	K	–	–	–	–	K	–
TC-QE-17	–	–	–	K	–	–	–	–	K	–
TC-QE-18	–	–	–	K	–	–	–	–	K	–
TC-QE-19	–	–	–	K	–	–	–	–	K	–
TC-QE-20	–	–	–	K	–	–	–	K	–	–
TC-QE-21	–	–	–	K	–	–	–	K	–	–
TC-QE-22	S	S	S	K	K	K	–	–	–	S
TC-QE-23	S	S	S	K	K	S	–	–	–	S
TC-QE-24	S	S	S	K	K	S	–	–	–	K
TC-QE-25	S	S	S	K	K	S	–	–	–	K
TC-QE-26	S	S	S	K	K	S	–	–	–	S
TC-QE-27	S	S	S	K	K	K	–	–	–	K
TC-QE-28	K	K	K	K	K	S	–	–	–	K
TC-QE-29	K	K	K	K	S	K	–	–	–	K
TC-QE-30	K	K	K	K	S	S	–	–	–	K
Kills	4	6	6	30	16	4	1	3	5	13

Discussion of the columns. The columns fall into three groups.

Boundary mutants (N1, N2, N3). N2 and N3, the relational mutants at the boundary of q_4, are killed exactly by the six tests that contain an input with $\Delta = 0$: TC-QE-05, -10, -15, -28 (its middle step), -29 and -30. They survive every other test, whatever the sign or magnitude of Δ. N1 ($b^2 - 4ac \to b^2 - 2ac$) is killed in this suite by TC-QE-05, -28, -29 and -30, the $\Delta = 0$ tests with $ac \neq 0$: there $b^2 = 4ac$, so $\Delta' = b^2 - 2ac = 2ac \neq 0$ (at TC-QE-05, $\Delta' = 4 - 2 = 2 > 0$). It survives TC-QE-10 and TC-QE-15, where $ac = 0$ and $\Delta' = \Delta = 0$, and every test with $\Delta \neq 0$, where the sign happens to be preserved (TC-QE-06: $\Delta = 25 \to \Delta' = 37$). This is a property of the suite, not of the mutant: N1 changes the output of other inputs with $\Delta \neq 0$, e.g. $(1, 2, 2)$, where $\Delta = -4$ becomes $\Delta' = 0$; the suite contains no such input.

Formula mutants (N6, N10). N6 ($b^2 \to 2b$) is killed only where the error moves Δ across 0: TC-QE-06 ($14 - 24 = -10$), TC-QE-22 ($-2 - 0 = -2$), TC-QE-27 ($6 - 8 = -2$) and TC-QE-29 ($8 - 16 = -8$). N10 ($4ac \to 4bc$) survives eight tests for two different reasons: where $a = b$ or $c = 0$ the mutation has no numeric effect (e.g. TC-QE-04 $(1, 1, 10)$, TC-QE-07 $(1, 5, 0)$), and at TC-QE-12 $(1, 2, -4)$ Δ changes from 20 to 36 but keeps its sign. It is killed by the other thirteen tests of Case A, including the tests with $b = 0$ and $c \neq 0$, where $4bc = 0$ turns $\Delta = -4ac \neq 0$ into 0 (TC-QE-08, -09, -13, -14, -24, -25).

Branch mutants (N4, N5, N7–N9). These are killed by any test that reaches the mutated branch with an output the mutation changes. N4, which swaps the two classification algorithms, is killed by all 30 tests: no input yields the same output under the linear rules and under the quadratic rules with $\Delta = b^2$. N5 (strict sign labels swapped) is killed by the 16 tests that contain an input with $a \neq 0$ and $\Delta \neq 0$, and survives the five single-input tests with $\Delta = 0$. N7, N8 and N9 each change one sub-case of the linear branch and are killed exactly by the tests in that sub-case: N7 by TC-QE-03; N8 by TC-QE-02, -20 and -21; N9 by TC-QE-01 and TC-QE-16–19.

Every other test is - for them.

No quadratic mutant is equivalent, and each is killed, so the analytically expected score is 10/10.

Sensitivity to the nominal inputs. With the nominal inputs of Table 19, the ten L1 tests already kill all ten designed mutants (§5.13). For nine of them this follows from the structure of the table: N1–N3 are killed by the L1 test of QR5, a rule defined by $\Delta = 0$, and N4, N5, N7–N10 by L1 tests of the branches they change. N6 is the exception: its only L1 kill is TC-QE-06 $(1, 7, 6)$, where $2b - 4ac = -10$ happens to be negative. With another nominal input of QR6 that is equally far from $\Delta = 0$, such as $(1, 9, 2)$ ($\Delta = 73$, $2b - 4ac = 10$), N6 survives all L1 tests, and L1 kills nine of the ten designed mutants.

5.13 Executed Validation

This subsection reports the executed stage of §3.6 for the quadratic solver.

Implementation decisions. Coefficients outside $[-100, 100]$ raise an argument-range error (QS2). For $a \neq 0$, the sign of Δ is classified by testing $\Delta < 0$ first, then $\Delta > 0$, and returning OneRepeatedRealRoot otherwise; this is the order under which N2 and N3 change exactly the inputs with $\Delta = 0$ (§5.12). If $\Delta = 0$ were tested first, N2 and N3 would be equivalent mutants, so their kill sets depend on this implementation choice.

Suite execution. All 111 quadratic tests pass: 31 designed-suite tests (the 30 test cases, with TC-QE-28 executed as two sequence checks), 19 supplementary tests (the extremes ± 100 and the out-of-range values ± 101 of each coefficient, and further values the design does not need), 2 decision table checks, 48 matrix-reproduction checks (including the six relational slips below), 3 layer-ablation checks, 6 properties and 2 exhaustive checks (Table 2).

Decision table checks. Over the test range, every one of the $201^3 = 8{,}120{,}601$ triples is evaluated against the condition entries of all ten rules of Table 21, without calling the solver, and the matching rule's output is compared with the solver's output. Every triple matches exactly one rule, and the solver returns that rule's output for all of them. Of the twelve columns of the full table with $a \neq 0$, exactly the five shown infeasible in §5.6 contain no triple; the other seven, and the four columns with $a = 0$, are all reached. Table 29 gives the number of triples per rule.

Table 29: Triples of the test range $[-100, 100]^3$ per decision rule

Rule	Output	Triples	Share
QR1	OneLinearRoot	40,200	0.50 %
QR2	NoSolution	200	0.0025 %
QR3	InfinitelyManySolutions	1	0.00001 %
QR4	NoRealRoots	3,003,112	36.98 %
QR5	OneRepeatedRealRoot	904	0.011 %
QR6	TwoDistinctRealRoots	4,995,984	61.52 %
QR7	TwoDistinctRealRoots	40,000	0.49 %
QR8	NoRealRoots	20,000	0.25 %
QR9	TwoDistinctRealRoots	20,000	0.25 %
QR10	OneRepeatedRealRoot	200	0.0025 %
Total		8,120,601	100 %

The counts agree with the structure of the table: QR7 has $200 \cdot 200$ triples ($a, b \neq 0$, $c = 0$); QR8 and QR9 split the $200 \cdot 200$ triples with $b = 0$, $a, c \neq 0$ evenly by the sign of ac; QR10 has

the 200 triples $(k, 0, 0)$. Summed by output, the rules reproduce the distribution of Table 31.

Matrix reproduction. The analytical matrix of Table 28 was derived by substitution, without executing the mutant models, and is compared with the executed kill sets. Derivation and execution agree in 298 of the 300 cells. The two disagreements are the cells of N1 at TC-QE-05 $(1, 2, 1)$ and at TC-QE-28 (middle step $(1, 2, 1)$): the derivation gave `S`, but N1 turns $\Delta = 0$ into $\Delta' = 2 > 0$ there, so both cells are `K`; the table shows the executed values. With them, all ten columns coincide with the executed kill sets, and the executed score is 10/10.

Layer ablation. Table 30 reports the contribution of each cumulative layer; the last column gives the designed mutants killed by each layer on its own.

Table 30: Quadratic layer ablation: mutants killed by the designed tests of the cumulative layers

Layers	Tests	Designed (of 10)	Stryker (of 21)	Designed mutants killed by the added layer alone
L1	10	10	14	L1: N1–N10
L1–L2	27	10	14	L2: N2–N6, N8–N10
L1–L3	30	10	15	L3: N1–N6, N10
L1–L4	30	10	15	L4: none (no L4 test)

The Stryker column was measured as for the ATM (§4.13). The decision table layer kills all ten designed mutants and 14 of the 21 tool-generated ones; 15 is the maximum the designed suite can reach, since the other six remove the range check of one coefficient each. The boundary layer adds no kill on either population; neither population contains a relational slip at the zero tests, at which its tests of q_1–q_3 are aimed (see below). The one tool-generated mutant that L1 and L2 leave alive replaces $4ac$ by $(4/a)\,c$ in integer arithmetic, which differs from $4ac$ whenever $|a| \geq 2$ and $c \neq 0$. Among the designed inputs it changes the output only at TC-QE-29 $(2, 4, 2)$, where $\Delta' = 16 - 4 = 12$ instead of 0; TC-QE-08 and TC-QE-09 also have $|a| = 2$, but there Δ keeps its sign. The L3 test therefore adds it. The supplementary test with $a = 100$ also kills it. On the designed mutants the cumulative table shows no gain after L1, because L1 already kills all ten; the per-layer column shows that L2 alone and L3 alone would kill eight and seven of them. The L1 row uses the refined table; the L1 tests of the six rules of the minimal table (QR1–QR6) also kill all ten designed mutants.

Relational slips at the zero tests. No mutant of either population replaces one of the zero tests of the case split by a relational comparison (§5.11). Six such slips were therefore executed against the designed suite as additional mutants, each replacing one of $a = 0$, $b = 0$, $c = 0$ by ≤ 0 or ≥ 0. Four of them are killed by L1 tests: $a \geq 0$ by every L1 test with $a > 0$, $b \geq 0$ by TC-QE-01 $(0, 2, -4)$, $c \geq 0$ by TC-QE-02 $(0, 0, 5)$, and $a \leq 0$ by TC-QE-09 $(-2, 0, 3)$, the one L1 test with $a < 0$, a consequence of its nominal value. The other two survive every L1 test and are killed only by L2 tests: $b \leq 0$ by TC-QE-16 and TC-QE-18, the neighbours $b = -1$ of QR2 and QR3, and $c \leq 0$ by TC-QE-20, the neighbour $c = -1$ of QR3. These two faults are detected by the boundary layer and by no other layer. This does not depend on the nominal inputs: the only L1 tests that reach the tests of b and c with a nonzero value are TC-QE-01 (b) and TC-QE-02 (c), so whatever sign their nominal values have, the slip in the opposite direction survives L1, and the neighbours ± 1 of L2 kill both directions.

Tool-generated mutants. Stryker.NET generated 24 mutants; three were ignored by the tool as redundant block removals, no mutant failed to compile, and all other 21 were killed (0 survived). This 100 % score is the score of the complete test project; the designed suite alone

kills 15 of the 21 (71 %). The six it does not kill each remove one of the six coefficient-range checks; the specification has no such range (QS2), so no designed test can reach them, and they are killed by the supplementary tests. The two relational mutants at $\Delta = 0$ ($\Delta < 0 \to \Delta \leq 0$ and $\Delta > 0 \to \Delta \geq 0$, the tool-generated counterparts of N3 and N2) are killed by the designed suite.

The form of the implementation affects which mutants the tool can produce. If the sign of Δ is classified with a relational pattern match (`< 0`, `> 0`, default), the boundary mutants of that form (`<= 0`, `>= 0`) make the default arm unreachable, which the C# compiler rejects; the tool then reports them as compile errors and excludes them from the score, so that a score of 100 % no longer covers the $\Delta = 0$ boundary. The implementation classifies the sign with if-statements, for which those mutants compile. A measured mutation score is only as informative as the set of mutants the tool manages to compile.

Exhaustive execution of the test range. The classifier is executed on all 8,120,601 triples. It never fails, it agrees with a re-statement of the specification in arbitrary-precision arithmetic, and $\Delta \bmod 4 \in \{0, 1\}$ for every triple (§5.8). Table 31 gives the resulting distribution by output.

Table 31: Output classes over the test range $[-100, 100]^3$

Output	Triples	Share
TwoDistinctRealRoots	5,055,984	62.26 %
NoRealRoots	3,023,112	37.23 %
OneLinearRoot	40,200	0.50 %
OneRepeatedRealRoot	1,104	0.0136 %
NoSolution	200	0.0025 %
InfinitelyManySolutions	1	0.00001 %
Total	8,120,601	100 %

Two claims of this case study can be stated quantitatively over the test range. First, the reachability argument of §5.8 holds: among the 8,080,200 quadratic triples ($a \neq 0$), discriminants in $[-4, 4]$ occur only as -4 (338 triples), -3 (252), 0 (1,104), 1 (1,880) and 4 (1,754); $\Delta \in \{-2, -1, 2, 3\}$ never occurs. Second, the boundary $\Delta = 0$ occupies 1,104 of those 8,080,200 triples (0.0137 %), 904 of them in QR5 and 200 in QR10. A test selected uniformly at random from the quadratic triples hits it with probability 1.37×10^{-4}, so a random suite of 1,000 such tests contains no $\Delta = 0$ case with probability ≈ 0.87, and would then kill neither N2 nor N3, which differ from the specification only at $\Delta = 0$. The specification itself names $\Delta = 0$ as a case of its own, and the decision table turns it into a rule, so the coverage criterion requires a test there.

5.14 Case Study Discussion

The quadratic case study shows a second kind of derived condition. In the ATM, derived conditions are sums and shifted thresholds; here the central condition is the sign of a quadratic form, its boundary $\Delta = 0$ is a curved surface in three dimensions, and its integer neighbours are not all reachable (§5.8). The usual ± 1 neighbours of BVA are therefore not used for q_4; the nearest reachable values -3 and $+1$ take their place.

The decision table played a different role than in the ATM. The minimal table read from the specification has six rules, and one of them, $\Delta = 0$, is already a boundary: its coverage test is a boundary test. The refined table, an extension of this report, splits Case A by the coefficient conditions $b = 0$ and $c = 0$; its feasibility analysis shows how they constrain Δ (QR7, QR10) and gives the boundaries $b = 0$ and $c = 0$ rules inside Case A from which neighbours are taken. With the chosen nominal inputs, the L1 tests kill all ten designed mutants; one of these kills

(N6) depends on the nominal value of QR6 (§5.12). On the tool-generated mutants, L1 kills 14 of the 15 that the designed suite can reach; the remaining one, which replaces $4ac$ by $(4/a)\,c$, is killed only by the L3 test TC-QE-29 $(2, 4, 2)$, the one designed input on $\Delta = 0$ with $|a| \geq 2$. The boundary layer adds no kill on either population, but neither population contains a relational fault at the zero tests that the boundary tests of q_2 and q_3 target. Of six such slips executed separately, $b \leq 0$ and $c \leq 0$ survive every L1 test and are killed only by L2 tests (§5.13).

6 Discussion

6.1 Comparison of the Two Case Studies

Domains. The ATM has three integer inputs, two unbounded and one (*dailyUsed*) bounded below by 0, and two small categorical ones; the quadratic solver has three unbounded integer inputs, tested within $[-100, 100]^3$. Both output domains are flat (five and six labels), but the quadratic one is reached through a two-level case split.

Decision tables. The ATM table has one extended-entry condition over a single input (the amount classes) and two derived conditions over several inputs; its infeasible columns come from a domain restriction ($dailyUsed \geq 0$), and its don't-care entries from a fixed rule precedence. The quadratic table has one extended-entry derived condition (the sign of Δ); its infeasible columns come from algebraic dependence between conditions, and its only don't-care entry from the irrelevance of c in the linear case. Both tables are finer than the minimal table on purpose: the ATM keeps the three invalid amount classes apart (7 rules minimal, 9 used), and the quadratic table splits Case A by $b = 0$ and $c = 0$ (6 minimal, 10 used). In both, the rule count closes over the feasible columns (56 of 64; 11 of 16). The exhaustive sweep confirmed completeness and non-overlap of the quadratic table; for the ATM, the executed witnesses check the implementation against the table, not the table itself.

Boundaries. Table 32 lists the conditions and boundaries present in each case study.

Table 32: Conditions and boundaries present in each case study

Kind	ATM	Quadratic
Range and granularity boundaries	$amount \in \{50{,}000,\ 20{,}000{,}000\}$, lattice step 50,000	–
Equality conditions (boundary 0)	–	$a = 0$, $b = 0$, $c = 0$
Derived boundaries	$balance = amount + fee$; $dailyUsed + amount = 20{,}000{,}000$	$\Delta = b^2 - 4ac = 0$
Reachability restriction	lattice of valid amounts	$\Delta \equiv 0, 1 \pmod 4$
Categorical conditions (no boundary)	status, internal/external	–
Evaluation order	four-rule precedence (don't-care entries)	–
Domain edges without behaviour change	$dailyUsed = 0$	test range ± 100

Boundary interactions. In the ATM, interaction is additive: a threshold shifts by a constant (the fee), and two independent sums can be tight at once. In the quadratic solver, interaction is non-linear: the distance from the boundary depends jointly on all three coefficients. In both, a boundary test of a derived condition had to be constructed by solving the condition for one input given the others.

Mutation analysis. ATM mutants are dominated by relational operators, constants and evaluation order, reflecting a rule-based specification; quadratic mutants are dominated by formula changes and classification swaps. In both, the mutants that misbehave only at an equality boundary ($balance = T_5$ for M1; $\Delta = 0$ for N2, N3) are killed only by tests that lie on that boundary.

Executed results. Executed against an implementation, both designs behaved alike in three respects: the decision table checks agreed with the rule counts and infeasibility claims, the analytical matrices agree with execution (in all ATM cells and in 298 of 300 quadratic cells), and the only tool-generated mutants that the designed suites do not kill remove input checks outside the specified domain. They differ in where the designed mutants are killed. In the ATM, decision table coverage kills 3 of 13, the boundary layer 7 more, and the don't-care expansion and mutation feedback 3 more; on the seven mutants that the boundary layer adds, a size-matched random baseline kills 0.06 on average. In the quadratic solver, decision table coverage already kills all 10 with the chosen nominal inputs: N1–N3 because the rule QR5 is itself a test on $\Delta = 0$, N4, N5 and N7–N10 because any test of the branch they change kills them, and N6 only through the nominal value $(1, 7, 6)$.

6.2 Lessons Common to Both Case Studies

From decision rules to boundaries. A decision table states which combinations of conditions must be tested; it does not say which input within a combination to use. A nominal input passes whether or not the boundaries of its rule are implemented correctly, so it cannot show that they are. In the ATM, the nine nominal tests leave ten of the thirteen designed mutants alive, and nineteen randomly chosen interior members of the same rules leave on average 8.655 alive, where the nineteen boundary tests leave 3. In these data the gain is that of boundary inputs over interior inputs, not of L2 over L1: L2 alone kills the same ten designed mutants as L1–L2. The two techniques divide the work: the table supplies the rules and their conditions, BVA supplies the inputs at which each condition changes its value. Where a rule is itself defined by an equality, as $\Delta = 0$ is, its coverage test is already a boundary test.

Derived conditions need constructed tests. Both case studies contain conditions that are functions of several inputs ($balance \geq amount + fee$; the sign of $b^2 - 4ac$). For such a condition $\phi(x, y)$, the boundary $\phi = 0$ is a lower-dimensional subset of the joint input space; varying x alone, with y at a nominal value, meets it in at most finitely many points, and on the integers possibly in none (for $\Delta = 0$, only when $4ac$ is a perfect square; §5.9). Writing the derived condition as one condition of the decision table, and taking its boundary values from the derived quantity, places the tests on it. This is the setting of domain testing [10], whose ON points lie on a border and OFF points next to it; the $T - 1$, T, $T + 1$ items of c_3 and c_5 play these roles for linear borders, and $\Delta = 0$ is a non-linear one.

Don't-care entries hide behaviour. A collapsed rule asserts that some conditions do not matter. When that assertion rests on an evaluation order, testing it requires setting the don't-care entry to the value that would select another rule. In the ATM, the two modelled precedence reversals (M11, M14) are killed by such tests and by no test of the coverage criterion or the boundary layer; the random baseline kills them in 50 % and 78.5 % of its draws precisely because it fills don't-care entries at random, often with failing values. The other four transpositions of two checks, including the adjacent R2/R3, are exposed even by the coverage test of DR4, TC-ATM-04, whose input fails R2, R3 and R5 together; for R1/R5 this depends on its nominal amount (§4.13). This is why the don't-care expansion is reported as an extension of decision table testing rather than omitted.

A one-unit step can change the output category. A derived quantity can range over many values while the output stays constant, and then change category at one step. *dailyUsed*+*amount* yields an approval-eligible result for every value up to 20,000,000, and Rejected_DailyLimit at 20,000,001. In the quadratic case, the reachable sequence $\Delta = -3 \to 0 \to 5$ (TC-QE-28) changes the output twice, NoRealRoots → OneRepeatedRealRoot → TwoDistinctRealRoots, under unit steps of b alone: no real intersection with the x-axis, then tangency, then two crossings. A suite that samples only values far from every boundary cannot observe such a change.

What the mutation results show, and what they do not. The layer ablations (Tables 17 and 30) report which mutants each layer kills in these two mutant populations. They are consistent with the premise of BVA that boundary tests detect faults that interior tests do not: in the ATM the boundary layer kills seven designed mutants beyond L1, while a random interior baseline of the same size kills on average 0.06 of the same seven; and in the quadratic solver the relational slips $b \leq 0$ and $c \leq 0$, which neither mutant population contains, are killed only by boundary tests. They do not establish that the same holds for other programs: the designed mutants were written with knowledge of the boundaries the suites target (Section 7), the tool-generated population is small, and the correlation between mutant detection and real-fault detection is weak once suite size is controlled [9].

7 Limitations

Internal validity.

- The mutant-killing claims of §4.12 and §5.12 were derived by substitution and confirmed by executing mutant models and one implementation written for this report. Substitution and execution agree in 816 of 818 cells; the two disagreements (§5.13) show that hand derivation alone is not reliable at this size.
- The designed mutants were written to probe the thresholds, the fee and the evaluation order of the ATM (M1–M14) and the $\Delta = 0$ boundary, the case split and the sub-case outputs of the quadratic solver (N1–N10). None of them changes the comparison at $b = 0$ or $c = 0$, so the quadratic boundary tests of q_2 and q_3 are measured only by the separately executed relational slips of §5.13. The L3b tests were constructed to make exactly the rule pairs of M11 and M14 fail, and the L4 test exists because L1–L3 do not kill M8; these kills are by construction. The full analytically expected scores (13/13, 10/10) are therefore partly a designed-in property of the construction, not an unbiased comparison of boundary tests with interior tests.
- The six relational slips (§5.13) and the six precedence transpositions (§4.13) were also chosen by the designer of the suites, to probe the zero tests and the precedence. Each set covers one fault class completely (both directions at each of the three zero tests; every transposition of two checks), but neither is an independent sample of faults.
- The tool-generated mutants were not constructed together with the suites, but the implementation was. Two of its forms affect which mutants the tool produces: the if-statements that classify the sign of Δ (§5.13) and the rearranged form of R3 (§4.13). The population is small (22 and 21 killable mutants) and contains no operator that reorders rules or changes the modulus, so it cannot measure the contribution of L3b and L4.
- The 100 % Stryker scores are scores of the complete test projects, which include supplementary tests, properties and, for the quadratic solver, the exhaustive sweep. The designed suites alone reach 21 of 22 (95 %) and 15 of 21 (71 %).

- The kill sets of N2 and N3 depend on the order in which the implementation tests the sign of Δ (§5.13). The quadratic L1 result depends on one nominal value (N6, §5.12), and the ATM L1 result on the balances chosen for DR8 and DR9 and, for the transposition R1/R5, on the amount chosen for DR4 (§4.13).
- The random baseline uses one sampling scheme, stated in §4.13; other schemes give other distributions. It is computed for the designed ATM mutants only: none was computed for the quadratic solver, and none for the tool-generated mutants, which are only available through Stryker runs.
- The per-layer Stryker scores were obtained by restricting the designed suite through an environment variable; the other test groups stayed in the run and were excluded from the count by test name. The Microsoft Testing Platform runner of Stryker.NET was marked as a preview at the time of the run.
- Equivalence of M4 was argued analytically and checked by execution over 20,300,001 amounts. For the other mutants, kill status was established against the suite inputs only; no search was made for further distinguishing inputs, and N1, for example, changes the output of inputs that the suite does not contain (§5.12).
- The implementation, the executable mutant models and the decision table oracles were written by the same party that designed the suites, with AI assistance, so the executed results test the consistency of design, models and implementation with the stated specification, not the specification's adequacy for a real ATM or solver. The arbitrary-precision oracle of the exhaustive sweep re-states the same formula as the implementation, so it detects overflow and branch-order errors but is not an independent check of the formula.

Construct validity.

- The don't-care expansion was applied only to the rule evaluated next in the precedence order, and only two reversals are modelled as designed mutants; the other four pairwise transpositions of the checks were executed separately (§4.13). Expanding every don't-care entry would restore the full table (56 feasible ATM columns) and might expose faults in orderings that move more than two rules, which are not analysed here.
- Two conditions, c_5 and q_4, are derived from computed values (the fee; the discriminant), and both decision tables are refined beyond the minimal table (§4.6, §5.6). These are modelling choices of this report; a table built only from conditions on raw inputs, or the minimal table, would have different columns and a different L1 layer.
- The assumptions AS1–AS3 and QS1–QS2 settle points that the problem statements leave open. AS2 is what makes eight ATM columns infeasible; an accumulator that also subtracts reversed transactions could produce a negative *dailyUsed*, which this report does not analyse.

External validity. Both specifications are small, use integer arithmetic only, and were implemented once; no real faults were available. The observations about which layer kills which mutants therefore describe these two case studies and their mutant populations, and are not claims about decision table testing and BVA in general.

8 Conclusion

This report applied decision table testing, boundary value analysis and mutation analysis to two specifications with multi-variable behaviour, a rule-based ATM validator and an algebraic

classifier, and validated both designs by execution. The research questions of Section 1 can be answered as follows, within the limits of Section 7.

RQ1. Conditions over several inputs were kept as conditions of the table (the daily sum, the fee-adjusted balance, the sign of Δ), and single-input conditions with several outcomes were written as extended-entry conditions whose entries partition the input's domain. The infeasible columns followed from a domain assumption in the ATM and from algebraic dependence in the quadratic solver, and were proved. The rule counts close over the feasible columns (56 of 64 for nine ATM rules; 11 of 16 for ten quadratic rules); this is a necessary condition. For the quadratic table, the exhaustive sweep of all 8,120,601 triples of the test range confirmed completeness and non-overlap, including the absence of inputs in infeasible columns; for the ATM, the executed witnesses check the implementation against the table. Both tables are refinements of minimal tables of 7 and 6 rules; the quadratic refinement by $b = 0$ and $c = 0$ exposes two rules, QR7 and QR10, in which a zero coefficient fixes the sign of Δ.

RQ2. 3-value BVA added 19 ATM and 17 quadratic tests. In the ATM, decision table coverage alone killed 3 of 13 designed and 14 of 22 tool-generated mutants; with the boundary layer the counts rose to 10 and 21, while a random baseline of the same size from the same rules killed 4.345 designed mutants on average, and 0.06 of the seven that the boundary layer adds. The added kills were the relational mutants at equality boundaries and the mutants that shift a threshold, including the fee, which is visible only when the balance condition is analysed in the external column. In the quadratic solver, decision table coverage already killed all ten designed mutants with the chosen nominal inputs: N1–N3 because the rule QR5 is itself a test on $\Delta = 0$, the others because any test of the branch they change kills them (N6 only through its nominal input). The boundary layer added no kill on either population, but neither population contains a relational slip at the zero tests, at which its tests are aimed; of six relational slips at these zero tests, executed separately, two ($b \leq 0$, $c \leq 0$) survive one test per rule and are killed only by boundary tests.

RQ3. The two modelled precedence reversals (M11, M14) were killed only by the don't-care expansion, which lies outside the coverage criterion of decision table testing; the other four transpositions of two checks, executed separately, are already exposed by the coverage test of DR4, whose input fails three rules at once (for R1/R5 only through its nominal amount). One designed mutant, a wrong modulus (M8), needed a test from mutation feedback, and a related wrong modulus, 25,000, survives the whole suite. The boundary combinations added no kill in either case study, except one tool-generated quadratic mutant, which was killed only by a further point on $\Delta = 0$. The tool-generated mutants contain no reordering or modulus fault, so the contribution of L3b and L4 is shown only on designed mutants, by construction.

Across both case studies, every test case answers the question "why does this test exist?" by pointing to a decision rule, a boundary coverage item, a boundary combination or further point on a derived boundary, a don't-care expansion, or a mutant that no earlier test kills, and the executed results report what each of these sources contributes, including the cases in which it contributes nothing.

Future work. Three extensions would test how far these observations generalise. First, the layer ablation should be repeated on larger mutant populations that the suites were not designed against, for example mutants generated from several independent implementations, with operators that reorder statements, and with size-matched baselines for the tool-generated mutants as well. Second, the don't-care expansion was limited to the rule evaluated next; expanding all don't-care entries would show whether orderings that move more than two rules hide further faults, and the granularity condition should be tested against every divisor class of the step (here 60,000 and 75,000). Third, the method should be applied to specifications with more interacting conditions, where the full table is too large to enumerate and the choice of which columns to expand becomes a design decision in its own right.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The implementation, all test code, the executable mutant models and decision table oracles, the Stryker.NET configuration and the continuous-integration workflow that re-runs the tests and the mutation analysis are available in the public repository `https://github.com/thnhmai06/SWT-Blackbox`. §3.6 lists the tool versions used; the environment variable `SWT_LAYERS` restricts the designed suite to given layers for the layer ablation, and `tools/StrykerDesignedKills.cs` counts the mutants that the designed suite kills in a Stryker report.

CRediT authorship contribution statement

Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Visualization, Writing – original draft, Writing – review & editing: Mai Hải Thành.

Declaration of generative AI use

During the preparation of this manuscript, generative AI assistance (Claude, Anthropic) was used to help structure the author's notes into an academic paper, to draft and revise the decision table, boundary value analysis and mutation testing sections, to cross-check mathematical and test-design reasoning, to review the manuscript against a checklist, and for language editing. The analytical Test×Mutant matrices were derived by substitution with the help of AI models that were instructed not to read or execute the mutant models or any other code; two of the 818 derived cells disagree with execution and are reported in §5.13. For the executed validation, AI coding assistants (Claude, Anthropic, and models accessed through the OpenCode agent) were used to help write the implementation, the test code, the mutant models and the continuous-integration configuration, and to run the test and mutation tools; all reported numbers are taken from the generated tool reports. The author reviewed and revised the AI-assisted material and takes full responsibility for the final content of this manuscript.

References

[1] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. John Wiley & Sons, Hoboken, NJ, 2011.

[2] B. Beizer, *Software Testing Techniques*, 2nd ed. Van Nostrand Reinhold, New York, 1990.

[3] P. C. Jorgensen and B. DeVries, *Software Testing: A Craftsman's Approach*, 5th ed. Auerbach Publications (CRC Press), Boca Raton, FL, 2021. Ch. 5 (Boundary Value Testing) and Ch. 7 (Decision Table Based Testing).

[4] L. Copeland, *A Practitioner's Guide to Software Test Design*. Artech House, Boston, MA, 2004. Ch. 4 (Boundary Value Testing), Ch. 5 (Decision Table Testing), Ch. 8 (Domain Analysis Testing).

[5] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on Test Data Selection: Help for the Practicing Programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[6] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[7] J. H. Andrews, L. C. Briand, and Y. Labiche, "Is Mutation an Appropriate Tool for Testing Experiments?," in *Proc. 27th International Conference on Software Engineering (ICSE)*, St. Louis, MO, 2005, pp. 402–411.

[8] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are Mutants a Valid Substitute for Real Faults in Software Testing?," in *Proc. 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, Hong Kong, 2014, pp. 654–665.

[9] M. Papadakis, D. Shin, S. Yoo, and D.-H. Bae, "Are Mutation Scores Correlated with Real Fault Detection? A Large Scale Empirical Study on the Relationship Between Mutants and Real Faults," in *Proc. 40th International Conference on Software Engineering (ICSE)*, Gothenburg, 2018, pp. 537–548.

[10] L. J. White and E. I. Cohen, "A Domain Strategy for Computer Program Testing," *IEEE Transactions on Software Engineering*, vol. 6, no. 3, pp. 247–257, 1980.

[11] T. J. Ostrand and M. J. Balcer, "The Category-Partition Method for Specifying and Generating Functional Tests," *Communications of the ACM*, vol. 31, no. 6, pp. 676–686, 1988.

[12] S. C. Reid, "An Empirical Analysis of Equivalence Partitioning, Boundary Value Analysis and Random Testing," in *Proc. 4th International Software Metrics Symposium (METRICS)*, 1997, pp. 64–73.

[13] K. Claessen and J. Hughes, "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs," in *Proc. 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, Montreal, 2000, pp. 268–279.

[14] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic Testing: A Review of Challenges and Opportunities," *ACM Computing Surveys*, vol. 51, no. 1, Art. 4, pp. 4:1–4:27, 2018.

[15] International Software Testing Qualifications Board (ISTQB), *Certified Tester Foundation Level Syllabus*, v4.0.1, 2024. §4.2.2 (Boundary Value Analysis) and §4.2.3 (Decision Table Testing).